PYTHON PROGRAMMING - ASSIGNMENT 4 (CO4)

ROBUST DATA PROCESSING WITH FILES, EXCEPTIONS, MODULES AND PACKAGES

Name:- Balachandru.P

Register Number:- 1912260301C

Slot:- C

Code & Course:- CSA08 - Python Programming

Faculty:- Pokuri Swetha

GitHub Repository Link:- https://github.com/balachandru4562bala-blip/Python-slot-c.git

Contents: Task 1 File-processing design review (6) | Task 2 Biometric attendance files (8) | Task 3 Exceptions in the attendance processor (9) | Task 4 File and exception forensics (7) | Task 5 The same utility organised four ways (7) | Task 6 College fest registration desk (13) | Total 50 marks

All programs were written as .py files and actually run; every output in this report is a real run on my computer (Python 3.10.12, standard library only). Timings change from run to run, so wherever I compare two designs I also give counts (lines read, files left open, files changed), because counts do not change. The repository for this assignment is CSA08-1912260301C-Assignment4, with one folder per task (Task01 to Task06).

TASK 1 - FILE-PROCESSING DESIGN REVIEW: DIAGNOSE AND REPAIR A POOR SCRIPT (6 Marks)

1. Aim

To find the design deficiencies in the hostel office's mess-bill script, to predict what it does on a small file before running it, to rewrite it safely (with statement, line by line, specific exceptions, a list of rejected lines, a function that takes the file name), and to measure why reading a 50,000-line file line by line is better than reading it whole.

2. Predictions made before running

I made a 7-line test file, Task01/mess_oct.txt (register number, name, amount). Five lines are fine, one has a letter amount, one has a missing field, and two good lines have paise:

Line	Content	Original script will...	Why
1	192311001,Arun,1250.50	drop it silently	int('1250.50') raises ValueError, bare except hides it
2	192311002,Bhavani,1800	count it	plain whole number
3	192311003,Charan,abc	drop it silently	int('abc') raises ValueError
4	192311004,Divya	drop it silently	parts[2] raises IndexError
5	192311005,Elango,2100	count it	plain whole number
6	192311006,Farida, 1500	count it	int() ignores the spaces around the number
7	192311007,Gokul,2250.75	drop it silently	paise again
end	(empty line after the last \n)	drop it silently	split('\n') gives '', parts[2] raises IndexError

Table T1-1: predictions for the original script

So I predicted Total 5400 (1800 + 2100 + 1500) with no message at all, while the correct total is 1250.50 + 1800 + 2100 + 1500 + 2250.75 = 8901.25. The office would be short by 3501.25 and would never know. I also predicted that on my laptop the script would not even start, because C:\Users\Office\Desktop does not exist here.

3. Task 1.1 - Diagnosis: the design deficiencies (2 marks)

#	Deficiency	What could go wrong in the hostel office
1	Hard-coded file path 'C:\\Users\\Office\\ Desktop\\mess_oct.txt'	The script only works on one computer and one month. On any other PC, or in November, it crashes with FileNotFoundError (my first run shows this) or, worse, silently totals last month's file.
2	File opened with open() and never closed	The file handle stays open until Python happens to clean up. On Windows the clerk may not be able to edit, rename or delete mess_oct.txt while the program is alive, and a long-running program can run out of handles.
3	Whole file read into memory with f.read()	For 50,000 lines the file is held as one big string and then split into 50,000 more strings. My measurement below: about 5.8 MB peak, against 21 KB for reading line by line.
4	Bare 'except: pass'	It hides every error, including typing mistakes in the code itself and even Ctrl+C. The script can never tell the clerk that something went

		wrong.
5	int() cannot read amounts with paise	Arun's 1250.50 and Gokul's 2250.75 were thrown away. In my run the total was short by 3501.25, and money was lost with no message. (float would also be a poor choice for money; I use Decimal.)
6	report = report + ... inside the loop	Every + builds a new, longer string and copies the old one. For 50,000 students that is a lot of copying; a list of lines joined once at the end is the normal way.
7	The last empty line of the file	split('\n') gives an empty string at the end. It causes an IndexError that the bare except hides, so a harmless empty line is treated exactly like a real damaged line and nobody can tell them apart.
8	No record of the rejected lines	The office cannot know which student's bill was skipped, how many, or why, so a student can be un-billed or double-billed after someone re-enters the data.

Table T1-2: eight deficiencies (the assignment asks for at least five)

4. Program

Files: Task01/task1_original.py (copied exactly from the assignment), Task01/task1_original_local.py (the same script with only the file path changed so it can run here), Task01/task1_repaired.py (my repair) and Task01/task1_alternative.py (the measurements).

```
# Task 1 - the hostel office's ORIGINAL script, copied exactly from the assignment.
f = open("C:\\Users\\Office\\Desktop\\mess_oct.txt")
data = f.read()
total = 0
report = ""
for line in data.split("\n"):
    try:
        parts = line.split(",")
        total = total + int(parts[2])
        report = report + parts[1] + " " + parts[2] + "\n"
    except:
        pass
print(report)
print("Total", total)
```

```python
"""Task 1.2 - repaired mess-bill totaller (with statement, line by line,
specific exceptions, rejected-lines list, file name as a parameter)."""
import sys
from decimal import Decimal, InvalidOperation


def total_mess_bills(path):
    """Return (total, report_lines, rejected). rejected holds (line number, text, reason)."""
    total = Decimal("0")
    report_lines = []
    rejected = []
    with open(path, encoding="utf-8") as f:          # closed automatically, even on error
        for number, line in enumerate(f, start=1):   # one line in memory at a time
            line = line.strip()
            if line == "":                           # the empty last line is not an error
                continue
            parts = [p.strip() for p in line.split(",")]
            try:
                if len(parts) != 3:
                    raise ValueError(f"expected 3 fields, found {len(parts)}")
                amount = Decimal(parts[2])           # accepts 1250.50 (paise)
            except (ValueError, InvalidOperation):
                reason = f"expected 3 fields, found {len(parts)}" if len(parts) != 3 \
                    else f"amount {parts[2]!r} is not a number"
                rejected.append((number, line, reason))
                continue
            total += amount
            report_lines.append(f"{parts[1]} {amount}")
    return total, report_lines, rejected


def main(path):
    try:
        total, report_lines, rejected = total_mess_bills(path)
    except FileNotFoundError:
        print(f"Cannot find the file {path!r}. Check the name and the folder.")
        return 1
    except PermissionError:
        print(f"No permission to read {path!r}.")
        return 1
    except UnicodeDecodeError:
        print(f"{path!r} is not a UTF-8 text file.")
        return 1
    print("\n".join(report_lines))                   # join once, not repeated +
    print("Total", total)
    print(f"Rejected lines: {len(rejected)}")
    for number, text, reason in rejected:
        print(f"  line {number}: {text!r} -> {reason}")
    return 0


if __name__ == "__main__":
    sys.exit(main(sys.argv[1] if len(sys.argv) > 1 else "mess_oct.txt"))
```

5. Output

Real runs (Python 3.10.12). First the original script exactly as given, then with only the path changed:

```
Terminal - Python 3.10.12
$ cd Task01
$ python3 task1_original.py
Traceback (most recent call last):
  File "/home/sandbox/a4/CSA08-1912260301C-Assignment4/Task01/task1_original.py", line 2, in
    <module>
    f = open("C:\\Users\\Office\\Desktop\\mess_oct.txt")
FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Users\\Office\\Desktop\\mess_oct.txt'
```

```
Terminal - Python 3.10.12
$ cd Task01
$ python3 task1_original_local.py
Bhavani 1800
Elango 2100
Farida  1500

Total 5400
```

Repaired script on the same file, and with a file name that does not exist:

```
Terminal - Python 3.10.12
$ cd Task01
$ python3 task1_repaired.py
Arun 1250.50
Bhavani 1800
Elango 2100
Farida 1500
Gokul 2250.75
Total 8901.25
Rejected lines: 2
  line 3: '192311003,Charan,abc' -> amount 'abc' is not a number
  line 4: '192311004,Divya' -> expected 3 fields, found 2
```

```
Terminal - Python 3.10.12
$ cd Task01
$ python3 task1_repaired.py nofile.txt
Cannot find the file 'nofile.txt'. Check the name and the folder.
```

6. Analysis and Task 1.2 - the repair (3 marks)

- The original run matched my prediction: Total 5400, three bills shown, no message. The four silently lost lines were Arun, Charan, Divya and Gokul. Two of them (Charan and Divya) really are bad. Two (Arun and Gokul) are good lines the script could not read. The script treated all four the same. Farida's line passed only because int() ignores spaces, and the report line shows the extra space ('Farida 1500 ').
- The first run failed with FileNotFoundError because of the hard-coded path, as I predicted. That is deficiency 1 shown by a real run.
- Repair 1: with open(path, encoding='utf-8') as f closes the file when the block ends, even if an error happens inside. I also state the encoding, because the default on Windows depends on the computer.
- Repair 2: for number, line in enumerate(f, start=1) reads one line at a time and also gives me the line number for the rejected list.

- Repair 3: specific exceptions only. Inside the loop I catch (ValueError, InvalidOperation), which is what a wrong field count or a bad amount really raises. FileNotFoundError, PermissionError and UnicodeDecodeError are caught once, in main(), with a clear message. A typing mistake in my own code is not caught, so it shows up straight away instead of being hidden.
- Repair 4: Decimal(parts[2]) reads 1250.50 and 2250.75 correctly. The repaired total is 8901.25, exactly my hand calculation, and the empty line is skipped on purpose (not counted as an error).
- Repair 5: rejected is a list of (line number, text, reason). The run shows line 3 (amount 'abc' is not a number) and line 4 (expected 3 fields, found 2), so the clerk knows exactly which two students to look at.
- Repair 6: total_mess_bills(path) takes the file name as a parameter, so the same function works for October, November or a test file. The report is a list joined once with '\n'.join(...).

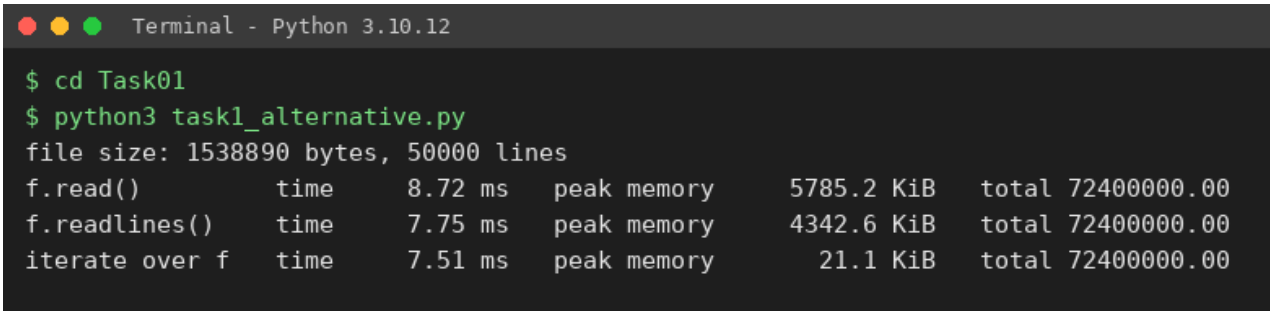
7. Task 1.3 - Design Defence (1 mark, 100 words or fewer)

My version is safer for a 50,000-line file because it reads one line at a time, so memory stays tiny instead of holding the whole file. The with statement always closes the file, even after an error. Specific exceptions mean only expected problems are handled; real mistakes still show. Decimal keeps paise exact, so totals are correct. Bad lines are not lost: each is stored with its line number and reason, so the office can fix them. Passing the file name makes the function reusable and testable. Nothing fails silently any more.

(92 words)

8. Alternative Concept and Complexity Analysis

Alternative: read the whole file with f.read() (then split) or f.readlines() and loop over the result, instead of iterating over the file. I wrote both in task1_alternative.py and measured them on a generated 50,000-line file (1,538,890 bytes). All three give the same total.



Version	Time (best of 5)	Peak memory measured	Time complexity	Space complexity
f.read()	8.72 ms	5,785 KiB	O(n)	O(file size): one big string, and split() makes a list of n strings as well
f.readlines()	7.75 ms	4,343 KiB	O(n)	O(file size): a list of n line strings
iterate over f	7.51 ms	21 KiB	O(n)	O(longest line): only one line at a time

Table T1-3: 50,000 lines, measured with time.perf_counter and tracemalloc on my computer

All three read each line once, so the time is O(n) in every case, and the measured times are close to each other (a few milliseconds; the small differences change from run to run, so I do not trust them). The memory is what changes: f.read() peaked at about 5.6 MB and f.readlines() at about 4.2 MB, but iterating

over the file stayed at about 21 KiB, roughly 274 times less. The read-everything versions have space O(file size), so a file ten times bigger needs ten times more memory. Iterating needs space O(one line) however long the file is. The in-memory versions are only worth using when I need to go over the data more than once or jump around in it.

9. Conclusion

The script's real problem was silence: a bare except turned unreadable amounts, empty lines and a missing file into 'nothing happened', and the total was wrong by 3501.25. The repaired version reads line by line, keeps paise exact, reports each rejected line with a reason, and uses about a 275th of the memory of f.read() on 50,000 lines.

TASK 2 - FROM REQUIREMENT TO FILE-PROCESSING DESIGN: BIOMETRIC ATTENDANCE FILES (8 Marks)

1. Aim

To design, before writing code, how the biometric text files will be read and the two output files written: the data model, the file modes and methods, the pseudocode for a whole month of daily files, and a trace on a small file with a damaged line. Then to implement the design, run it on 8 daily files that contain damaged lines, and compare it with the design that re-reads all files for every student.

2. Predictions made before running

I made the sample data myself (Task02/make_daily_files.py): 6 students, 8 daily files named 2026-10-01.txt to 2026-10-08.txt. I chose who is absent, and I put three damaged lines in on purpose (day 3 missing the status, day 5 status 'X', day 6 a date written 03-10-2026) plus a blank line on day 6. A damaged line is not counted as present or absent, so that student has one day less in the total. From the absent days I predicted:

Register	Absent days	Damaged line?	Predicted present/total	Predicted %
192311001	none	no	8/8	100.0
192311002	day 3 (but that line is damaged)	day 3 (missing status)	7/7	100.0
192311003	days 2, 3, 4 and 7	no	4/8	50.0 (below 75)
192311004	day 6 (day 5 is damaged, status X)	day 5 (status X)	6/7	85.7
192311005	days 1, 2, 6, 7	no	4/8	50.0 (below 75)
192311006	day 8 (day 6 line is damaged)	day 6 (bad date)	6/7	85.7

Table T2-1: predictions. Expected error log: 3 lines (the blank line is skipped, not an error).

3. Task 2.1 - Model the Data (2 marks)

Input file format. One text file per day, UTF-8, one line per student: register number, date (YYYY-MM-DD) and status, separated by commas, for example 192311003,2026-10-03,A. P means present and A means absent. A line is damaged if it does not have exactly 3 fields, if the register number is not all digits, if the date is not a real YYYY-MM-DD date, or if the status is not P or A. Blank lines are ignored.

Structure	Used for	Why this one
counts = {register: [present, total]}	accumulating the attendance of every student while all files are read once	A dictionary finds a student by register number in constant time and creates the entry when the student appears first, so I do not need the student list in advance. The value is a list so I can change the two counts in place.
errors = [(file name, line number, text, reason), ...]	damaged lines	A list keeps them in the order found. A tuple per entry because an entry is never changed after it is written.
sorted(Path(folder).glob('*.txt'))	the list of daily files of the month	pathlib gives the files; sorted() makes the order the same on every computer (day 1 to day 8).
low = [(register, percentage), ...]	students below 75%	filled while writing the report.

Table T2-2: data structures

Output file 1: attendance_report.txt - a header line, then one line per student: Register, Present, Total, Percentage (one decimal), and at the end a section 'Below 75%:' with the students under 75%. Output file 2: errors.log - one line per damaged line in the form 'file name, line N: reason | the original text', for example 2026-10-05.txt, line 4: invalid status 'X' | 192311004,2026-10-05,X.

4. Task 2.2 - Design the File Operations (2 marks)

What	Mode	Encoding	Method	Why
each daily file	'r' (read)	utf-8	for line in f (iteration)	A month has many files and each can have many lines. Iteration keeps one line in memory and gives the line number with enumerate. read() or readlines() would hold a whole file at once; readline() is the same as iteration but needs a manual loop.
attendance_report.txt	'w' (write)	utf-8	f.write(text)	The monthly report is rebuilt from scratch every time. With 'a' a second run would append a second copy under the first.
errors.log	'w' (write)	utf-8	f.write(text)	The log describes this run of these files. If the office wanted a history of all runs I would change it to 'a'. It is written once at the end, after the list is complete.
listing the files	(not a file mode)	-	pathlib.Path(folder).glob('*.txt')	Finds the month's files without typing their names, and works the same on Windows and Linux.

Table T2-3: file operations. I always give encoding='utf-8' because the default on Windows depends on the computer.

5. Task 2.3 - Design the Processing: Pseudocode (3 marks)

Pseudocode written in the SIMATS Engineering algorithm style (ALGORITHM / INPUT / OUTPUT / BEGIN ... END, numbered steps, <- for assignment).

```
ALGORITHM ProcessMonthlyAttendance
INPUT  : folder  - the folder holding the daily files of the month
OUTPUT : attendance_report.txt, errors.log
BEGIN
 1.  counts <- empty dictionary
 2.  errors <- empty list
 3.  files  <- sorted list of all '*.txt' files in folder
 4.  FOR EACH file IN files DO
 5.      TRY
 6.          OPEN file for reading
 7.          lineNo <- 0
 8.          FOR EACH line IN file DO
 9.              lineNo <- lineNo + 1
10.              IF line is blank THEN
11.                  CONTINUE with next line
12.              END IF
13.              CHECK line:  exactly 3 fields, register is digits,
14.                           date is a real date, status is P or A
15.              IF line is damaged THEN
16.                  ADD (file name, lineNo, line, reason) TO errors
17.                  CONTINUE with next line
18.              END IF
19.              IF register NOT IN counts THEN
20.                  counts[register] <- [0, 0]
21.              END IF
22.              counts[register].total <- counts[register].total + 1
23.              IF status = 'P' THEN
24.                  counts[register].present <- counts[register].present + 1
```

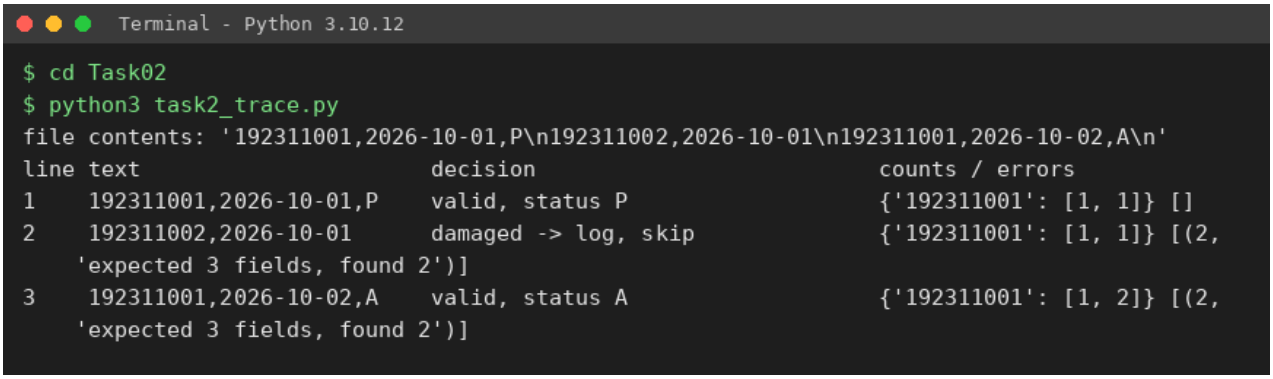
```
25.              END IF
26.          END FOR
27.          CLOSE file
28.      ON ERROR (file cannot be opened or read)
29.          ADD (file name, 0, 'file could not be read') TO errors
30.      END TRY
31.  END FOR
32.  OPEN attendance_report.txt for writing
33.  FOR EACH register IN counts (in sorted order) DO
34.      percentage <- 100 * present / total
35.      WRITE register, present, total, percentage
36.      IF percentage < 75 THEN ADD register TO low
37.  END FOR
38.  WRITE the list low
39.  CLOSE attendance_report.txt
40.  OPEN errors.log for writing
41.  FOR EACH entry IN errors DO WRITE entry
42.  CLOSE errors.log
END ALGORITHM
```

6. Task 2.4 - Trace (1 mark)

Small file (3 lines, one damaged): 192311001,2026-10-01,P / 192311002,2026-10-01 / 192311001,2026-10-02,A. I traced the pseudocode by hand and then ran task2_trace.py, which prints the state after every line. The two agree.

Line	Text	Steps used	counts after the line	errors after the line
1	192311001,2026-10-01,P	13-14 ok, 19-20 new entry, 22 total=1, 23-24 present=1	{'192311001': [1, 1]}	[]
2	192311002,2026-10-01	13-14 only 2 fields, so 15-17 add to errors and CONTINUE	{'192311001': [1, 1]}	[(2, 'expected 3 fields, found 2')]
3	192311001,2026-10-02,A	13-14 ok, 22 total=2, status is A so step 24 is skipped	{'192311001': [1, 2]}	[(2, ...)] unchanged

Table T2-4: hand trace. After the loop: 192311001 has 1/2 = 50.0%, and errors.log gets one line.



7. Program

File: Task02/task2_attendance.py. check_line() decides if a line is damaged, process_month() reads all files once into the dictionary, write_outputs() writes the two files.

```python
"""Task 2 - monthly attendance processor for the biometric files.

Reads every daily/*.txt file once, accumulates present/total in a dictionary,
writes attendance_report.txt (percentages, students below 75%) and errors.log
(file name + line number of every damaged line)."""
from datetime import date
from pathlib import Path
import sys

STATUS = ("P", "A")


def check_line(line):
    """Return (register, date_text, status) or raise ValueError with the reason."""
    parts = line.strip().split(",")
    if len(parts) != 3:
        raise ValueError(f"expected 3 fields, found {len(parts)}")
    reg, day, status = (p.strip() for p in parts)
    if not reg.isdigit():
        raise ValueError(f"register number {reg!r} is not numeric")
    try:
        date.fromisoformat(day)
    except ValueError:
        raise ValueError(f"date {day!r} is not YYYY-MM-DD") from None
    if status not in STATUS:
        raise ValueError(f"invalid status {status!r}")
    return reg, day, status


def process_month(folder):
    """Return (counts, errors). counts[reg] = [present, total]; errors = [(file, line no, text, reason)]."""
    counts = {}
    errors = []
    for path in sorted(Path(folder).glob("*.txt")):          # list the month's files
        try:
            with open(path, "r", encoding="utf-8") as f:     # read mode, one line at a time
                for number, line in enumerate(f, start=1):
                    if line.strip() == "":
                        continue
                    try:
                        reg, day, status = check_line(line)
                    except ValueError as err:
                        errors.append((path.name, number, line.strip(), str(err)))
                        continue
                    entry = counts.setdefault(reg, [0, 0])
```

```
                    entry[1] += 1
                    if status == "P":
                        entry[0] += 1
        except (OSError, UnicodeDecodeError) as err:
            errors.append((path.name, 0, "", f"file could not be read: {err}"))
    return counts, errors


def write_outputs(counts, errors, report_path, error_path):
    low = []
    with open(report_path, "w", encoding="utf-8") as out:    # write mode: new report each run
        out.write("Register   Present  Total  Percentage\n")
        for reg in sorted(counts):
            present, total = counts[reg]
            pct = 100 * present / total
            out.write(f"{reg}  {present:7d}  {total:5d}  {pct:9.1f}%\n")
            if pct < 75:
                low.append((reg, pct))
        out.write("\nBelow 75%:\n")
        for reg, pct in low:
            out.write(f"  {reg}  {pct:.1f}%\n")
        if not low:
            out.write("  (none)\n")
    with open(error_path, "w", encoding="utf-8") as log:
        for name, number, text, reason in errors:
            log.write(f"{name}, line {number}: {reason} | {text}\n")
    return low


if __name__ == "__main__":
    folder = sys.argv[1] if len(sys.argv) > 1 else "daily"
    counts, errors = process_month(folder)
    low = write_outputs(counts, errors, "attendance_report.txt", "errors.log")
    print(open("attendance_report.txt", encoding="utf-8").read())
    print("errors.log:")
    print(open("errors.log", encoding="utf-8").read())
```

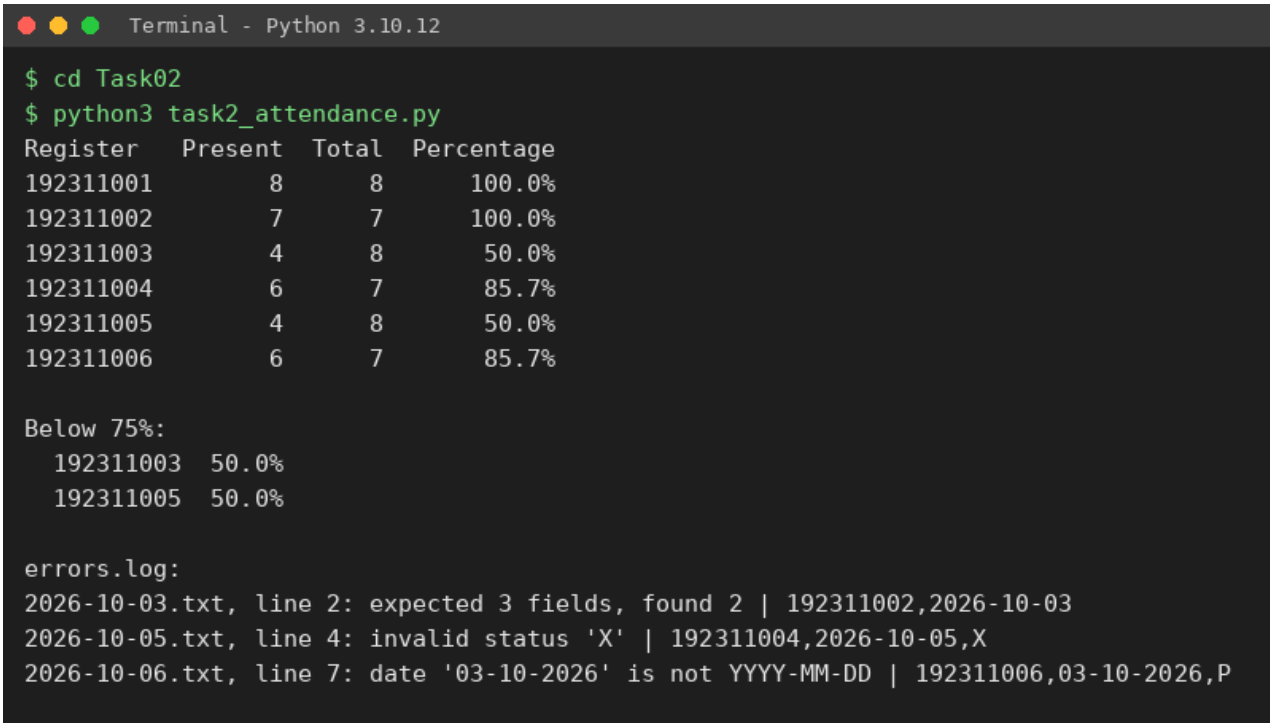
8. Output

Some of the input lines (day 3 has the damaged line 2; day 6 has the blank line and the bad date), then the real run of task2_attendance.py:

```
Terminal - Python 3.10.12

$ cd Task02
$ head -n 7 daily/2026-10-03.txt && echo ... && cat daily/2026-10-06.txt
192311001,2026-10-03,P
192311002,2026-10-03
192311003,2026-10-03,A
192311004,2026-10-03,P
192311005,2026-10-03,P
192311006,2026-10-03,P
...
192311001,2026-10-06,P
192311002,2026-10-06,P

192311003,2026-10-06,P
192311004,2026-10-06,A
192311005,2026-10-06,A
192311006,03-10-2026,P
```



9. Analysis

- All six percentages match my predictions in Table T2-1 (100.0, 100.0, 50.0, 85.7, 50.0, 85.7), and the two students below 75% are exactly 192311003 and 192311005.
- The error log has exactly the three damaged lines I planted, each with its file name and line number. The blank line on day 6 was skipped without an error, as designed. The line numbers are right: the date error on day 6 is line 7 because the blank line is line 3.
- A damaged line is not guessed. 192311002 has 7 days instead of 8 because day 3 was unreadable. I chose this because treating a damaged line as 'absent' would punish the student for a machine fault. The log lets the department fix the file and run again.
- Because process_month() reads each file inside its own try, one unreadable file adds one line to the log and the other files are still processed.

10. Alternative Concept and Complexity Analysis

Alternative: compute each student's attendance by re-reading all daily files for that student, instead of reading each file once and accumulating counts in a dictionary. With s students, f files and l lines per file this opens every file s times. I wrote both (task2_alternative.py), counted the lines read, and timed them. Both give the same percentages in every case.

```
Terminal - Python 3.10.12

$ cd Task02
$ python3 task2_alternative.py
   s   f    l | one pass reads   time s | per-student reads   time s | same answer
  10   5   10 |             50   0.0001 |               500   0.0004 | True
  20  10   20 |            200   0.0001 |              4000   0.0019 | True
  40  10   40 |            400   0.0002 |             16000   0.0048 | True
  80  10   80 |            800   0.0003 |             64000   0.0132 | True
  80  20   80 |           1600   0.0005 |            128000   0.0265 | True
```

Version	Lines read	Time complexity	Space used
one pass with a dictionary (my design)	f x l	O(f x l)	O(s) for the dictionary
re-read everything for every student	s x f x l	O(s x f x l)	O(1) extra (only one student's two counts)

Table T2-5: complexity. In my measurements the lines read were 50 against 500, 800 against 64,000 and 1,600 against 128,000.

Reading the numbers: the alternative always read exactly s times more lines than my design (10 times, 20, 40, 80 and 80 times in the five rows), so its work is multiplied by the number of students. Doubling the files from 10 to 20 (last two rows) doubled both counts (800 to 1,600 and 64,000 to 128,000), which is what O(f) says. The time column follows the line counts: for 80 students and 10 files the alternative needed 0.0132 s against 0.0003 s. The alternative saves a little memory (no dictionary of s entries), but the dictionary is tiny, one small list per student, so it is not worth the extra reading. For a real college (hundreds of students, about 30 files a month) the alternative would read hundreds of times more lines.

11. Conclusion

Designing the data model, the file modes and the pseudocode first made the program easy to write and every output predictable: 6 of 6 percentages and 3 of 3 damaged lines matched. Reading each file once into a dictionary is O(f x l) and needed only O(s) space, while re-reading for every student multiplies the work by the number of students.

TASK 3 - DESIGN THE ERROR HANDLING: EXCEPTIONS IN THE ATTENDANCE PROCESSOR (9 Marks)

1. Aim

To list everything that can go wrong when the attendance processor touches files, to define my own InvalidRecord exception and decide where it is raised and handled, to complete the given 'Complete the Code' program (TODO 1 and TODO 2), to show a full try / except / else / finally with raise ... from ..., and to show by experiment what a careless 'except Exception: pass' hides.

2. Predictions made before running

- Complete the Code: for the four lines in the assignment's data, line 1 (P) and line 2 (A) are good, line 3 has 2 fields, line 4 has status L. So summarise() should return (1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")]) exactly as the assignment says. With a file name that does not exist it should return (0, 0, [(0, 'file not found: ...')]).
- Processor: six test files (good, missing, permission denied, wrong encoding, empty, folder missing). I predicted that only the good file would return a count; the five others would each give one ProcessingError with the right built-in exception kept as the cause, and 'file closed' would be logged in every case, because of finally. The empty file has no built-in exception to keep.
- Misuse experiment: I predicted that with except Exception: pass and a typing error (presnt instead of present), the program would return (0, 0, []) for every file, which looks like a valid empty file.

3. Task 3.1 - Identify the Failures (3 marks)

Failure	Built-in exception	When it appears	How my design reacts
Missing folder (the path goes through a folder that does not exist)	FileNotFoundError	at open()	ProcessingError('file or folder does not exist') from the original; that file is skipped
Missing file	FileNotFoundError	at open()	same as above; the other files are still processed
Permission denied	PermissionError	at open()	ProcessingError('permission denied') from the original
Path is a folder, not a file	IsADirectoryError (a kind of OSError)	at open()	falls under OSError in Task 2's reader; logged
Wrong encoding (bytes that are not UTF-8)	UnicodeDecodeError	while reading, not at open()	ProcessingError('not UTF-8 text') from the original
Damaged line: wrong number of fields	unpacking a, b, c = parts gives ValueError; parts[2] gives IndexError	while parsing a line	my InvalidRecord('expected 3 fields, found N') raised by parse_line
Damaged line: wrong status letter	none from Python (the text is simply not P or A)	while parsing a line	my InvalidRecord('invalid status ...')
Damaged line: impossible date	ValueError from date.fromisoformat	while parsing a line	InvalidRecord('bad date ...') raised from the ValueError
Empty file	none: Python just reads zero	after reading	ProcessingError('is empty'), because an empty

	lines		attendance file is suspicious

Table T3-1: failure table. FileNotFoundError, PermissionError and IsADirectoryError are all subclasses of OSError; UnicodeDecodeError is a subclass of ValueError, so it must be written before ValueError in a chain of excepts.

4. Task 3.2 - Design a Custom Exception (2 marks)

class InvalidRecord(Exception) means 'this one line is not in the expected format'. I made it a separate class because a damaged line and an unreadable file need different reactions: a damaged line is skipped and logged, and the rest of the file is still good; an unreadable file stops that file only. If I used ValueError for both, a handler for one would catch the other by accident.

- Raised: in parse_line(line), the only function that knows the format. It raises InvalidRecord('expected 3 fields, found N'), InvalidRecord("invalid status 'L'") or InvalidRecord('bad date ...').
- Handled: one level up, in the loop of summarise() / process_file(), with except InvalidRecord as err. The handler stores (line number, message) and goes to the next line. It is not handled inside parse_line, because parse_line does not know the line number or where to store the error.
- Everything else (a typing error, a bug) is not handled there, so it cannot be mistaken for a damaged line.

5. Complete the Code

File: Task03/task3_complete_code.py. Only the two TODO places were replaced; the class, the docstrings, the data and the tempfile code are as given. One thing I decided: in the assignment's last lines, print(summarise(tmp.name)) and os.remove(tmp.name) must be outside the with block, because the written text is only guaranteed to be in the file after the with block closes it. The expected output is the first line of my run, character for character. For the second line I changed the file name to one that does not exist, and summarise() handles the FileNotFoundError itself.

```python
# Task 3 - "Complete the Code": TODO 1 and TODO 2 completed, rest of the given program unchanged.
import os
import tempfile


class InvalidRecord(Exception):
    """Raised when a line of the attendance file is not in the expected format."""


def parse_line(line):
    """'192311001,2026-10-08,P' -> ('192311001', '2026-10-08', 'P')"""
    parts = line.strip().split(",")
    # TODO 1: Raise InvalidRecord if there are not exactly 3 fields or the status is
    # not P or A; otherwise return the 3 fields as a tuple
    if len(parts) != 3:
        raise InvalidRecord(f"expected 3 fields, found {len(parts)}")
    if parts[2] not in ("P", "A"):
        raise InvalidRecord(f"invalid status '{parts[2]}'")
    return tuple(parts)


def summarise(path):
    present, absent, errors = 0, 0, []
    # TODO 2: Open the file with a with statement, parse each line inside try/except
    # InvalidRecord, count P and A, and record (line number, message) for bad lines
    try:
        with open(path, encoding="utf-8") as f:
            for number, line in enumerate(f, start=1):
                try:
                    reg, day, status = parse_line(line)
                except InvalidRecord as err:
                    errors.append((number, str(err)))
                    continue
                if status == "P":
                    present += 1
                else:
                    absent += 1
    except FileNotFoundError:        # the 'change the file name' part of the task
        errors.append((0, f"file not found: {os.path.basename(path)}"))
    return present, absent, errors


data = ("192311001,2026-10-08,P\n192311002,2026-10-08,A\n"
        "192311003,2026-10-08\n192311004,2026-10-08,L\n")
with tempfile.NamedTemporaryFile("w", suffix=".csv", delete=False,
                                 encoding="utf-8") as tmp:
    tmp.write(data)
print(summarise(tmp.name))
os.remove(tmp.name)

# Changed file name: a file that does not exist
print(summarise("no_such_attendance_file.csv"))
```

```
Terminal - Python 3.10.12

$ cd Task03
$ python3 task3_complete_code.py
(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])
(0, 0, [(0, 'file not found: no_such_attendance_file.csv')])
```

Expected output from the assignment: (1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")]) - my first line is identical.

6. Task 3.3 - The Complete try Statement (3 marks)

File: Task03/task3_processor.py. process_file() uses every part of the try statement. try opens and reads. Several specific excepts translate the built-in exceptions into one ProcessingError using raise ... from err, so the original (for example FileNotFoundError) is kept in __cause__. else runs only when nothing went wrong and logs success. finally always closes the file and logs it, whether the file was good, bad or missing.

```python
"""Task 3.2 / 3.3 - the attendance processor with the complete try statement.

Exceptions used:
  InvalidRecord      - my own exception, raised in parse_line() for a damaged line,
                       handled in process_file() (the line is logged and skipped).
  ProcessingError    - raised with 'raise ... from ...' when a whole file cannot be used;
                       the original built-in exception is kept in __cause__."""
from datetime import date


class InvalidRecord(Exception):
    """A single line is not 'register,YYYY-MM-DD,P|A'."""


class ProcessingError(Exception):
    """A whole file could not be processed."""

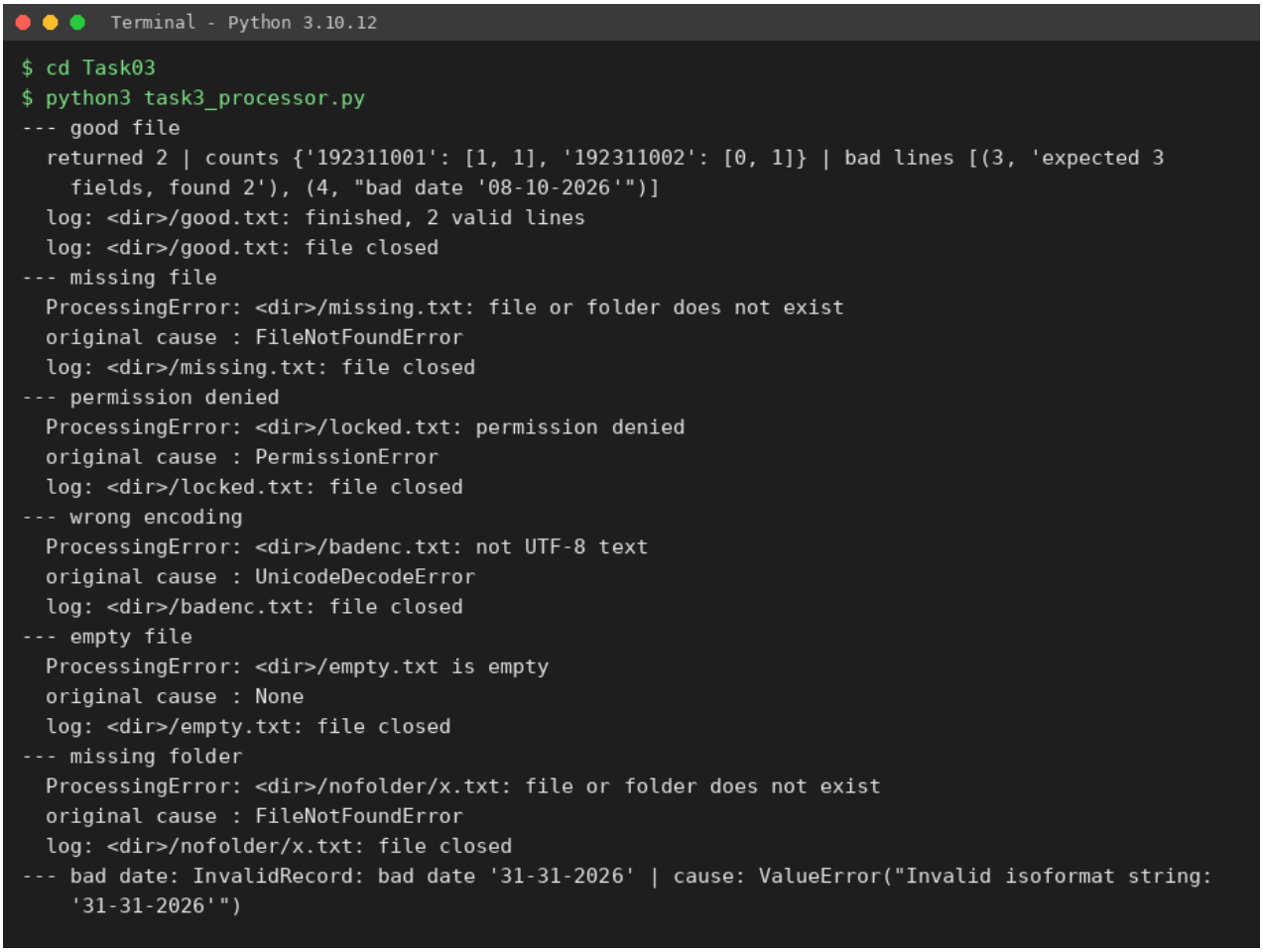

def parse_line(line):
    parts = line.strip().split(",")
    if len(parts) != 3:
        raise InvalidRecord(f"expected 3 fields, found {len(parts)}")
    reg, day, status = (p.strip() for p in parts)
    try:
        date.fromisoformat(day)
    except ValueError as err:
        raise InvalidRecord(f"bad date {day!r}") from err      # keeps the original cause
    if status not in ("P", "A"):
        raise InvalidRecord(f"invalid status {status!r}")
    return reg, day, status


def process_file(path, counts, bad_lines, log):
    """Add the file's valid lines to counts. Returns the number of valid lines."""
    valid = 0
    f = None
    try:
        f = open(path, encoding="utf-8")
        lines = f.readlines()
        if not lines:
            raise ProcessingError(f"{path} is empty")
        for number, line in enumerate(lines, start=1):
            try:
                reg, day, status = parse_line(line)
            except InvalidRecord as err:                       # handled here, one bad line only
```

```
                bad_lines.append((path, number, str(err)))
                continue
            entry = counts.setdefault(reg, [0, 0])
            entry[1] += 1
            entry[0] += status == "P"
            valid += 1
    except FileNotFoundError as err:
        raise ProcessingError(f"{path}: file or folder does not exist") from err
    except PermissionError as err:
        raise ProcessingError(f"{path}: permission denied") from err
    except UnicodeDecodeError as err:
        raise ProcessingError(f"{path}: not UTF-8 text") from err
    else:
        log.append(f"{path}: finished, {valid} valid lines")  # runs only if no exception
        return valid
    finally:
        if f is not None:
            f.close()                                        # always runs
        log.append(f"{path}: file closed")


if __name__ == "__main__":
    import os, stat, tempfile
    folder = tempfile.mkdtemp()
    def make(name, content, mode="w", **kw):
        p = os.path.join(folder, name)
        with open(p, mode, **kw) as fh:
            fh.write(content)
        return p
    good = make("good.txt", "192311001,2026-10-08,P\n192311002,2026-10-08,A\n192311003,2026-10-08\n192311001,08-10-2026,P\n")
    empty = make("empty.txt", "")
    badenc = make("badenc.txt", b"192311001,2026-10-08,P\n\xff\xfe\x00\n", mode="wb")
    locked = make("locked.txt", "192311001,2026-10-08,P\n")
    os.chmod(locked, 0)
    cases = [("good file", good), ("missing file", os.path.join(folder, "missing.txt")),
             ("permission denied", locked), ("wrong encoding", badenc), ("empty file", empty),
             ("missing folder", os.path.join(folder, "nofolder", "x.txt"))]
    for title, path in cases:
        counts, bad, log = {}, [], []
        print(f"--- {title}")
        try:
            n = process_file(path, counts, bad, log)
            print("  returned", n, "| counts", counts, "| bad lines", [(b[1], b[2]) for b in bad])
        except ProcessingError as err:
            print("  ProcessingError:", str(err).replace(folder, "<dir>"))
```

```
            print("  original cause :", type(err.__cause__).__name__ if err.__cause__ else None)
        for entry in log:
            print("  log:", entry.replace(folder, "<dir>"))
    # the raise ... from ... chain on a bad date
    try:
        parse_line("192311001,31-31-2026,P")
    except InvalidRecord as err:
        print("--- bad date: InvalidRecord:", err, "| cause:", repr(err.__cause__))
    os.chmod(locked, stat.S_IRUSR | stat.S_IWUSR)
```



7. Task 3.4 - Misuse Experiment (1 mark)

File: Task03/task3_misuse.py. I replaced the specific except clauses with except Exception: pass and introduced a typing error: presnt += 1 instead of present += 1. The same four lines of data were used.

```python
"""Task 3.4 - misuse experiment. Same summarise() logic, but the specific except clauses are
replaced by 'except Exception: pass' and a typing error is introduced in a variable name."""
import os, tempfile

data = ("192311001,2026-10-08,P\n192311002,2026-10-08,A\n"
        "192311003,2026-10-08\n192311004,2026-10-08,L\n")
path = os.path.join(tempfile.gettempdir(), "misuse.csv")
with open(path, "w", encoding="utf-8") as f:
    f.write(data)


def summarise_good(path):
    present, absent, errors = 0, 0, []
    with open(path, encoding="utf-8") as f:
        for number, line in enumerate(f, 1):
            parts = line.strip().split(",")
            if len(parts) != 3 or parts[2] not in ("P", "A"):
                errors.append((number, "bad"))
                continue
            if parts[2] == "P":
                present += 1
            else:
                absent += 1
    return present, absent, errors


def summarise_misused(path):
    present, absent, errors = 0, 0, []
    try:
        with open(path, encoding="utf-8") as f:
            for number, line in enumerate(f, 1):
                parts = line.strip().split(",")
                if len(parts) != 3 or parts[2] not in ("P", "A"):
                    errors.append((number, "bad"))
                    continue
                if parts[2] == "P":
                    presnt += 1          # typing error: the variable is called present
                else:
                    absent += 1
    except Exception:
        pass
    return present, absent, errors


print("correct code        :", summarise_good(path))
print("misused, with typo  :", summarise_misused(path))
print("misused, bad path   :", summarise_misused("no_such_file.csv"))
os.remove(path)
```

```
Terminal - Python 3.10.12

$ cd Task03
$ python3 task3_misuse.py
correct code        : (1, 1, [(3, 'bad'), (4, 'bad')])
misused, with typo  : (0, 0, [])
misused, bad path   : (0, 0, [])
```

What happened: the plain correct version returned (1, 1, [(3, 'bad'), (4, 'bad')]). The misused version returned (0, 0, []). The first line of data (a good P line) reached presnt += 1, which raises NameError because presnt does not exist. NameError is a kind of Exception, so the handler caught it, did nothing, and the whole loop ended on line 1. The function then returned its starting zeros. The result looks like a valid empty file, and a wrong file path gives exactly the same (0, 0, []), so I cannot tell a typing error, a missing file and an empty file apart. The error is hidden because except Exception catches almost every error, including my own bugs, and pass says to ignore it. With specific excepts (InvalidRecord, FileNotFoundError) the

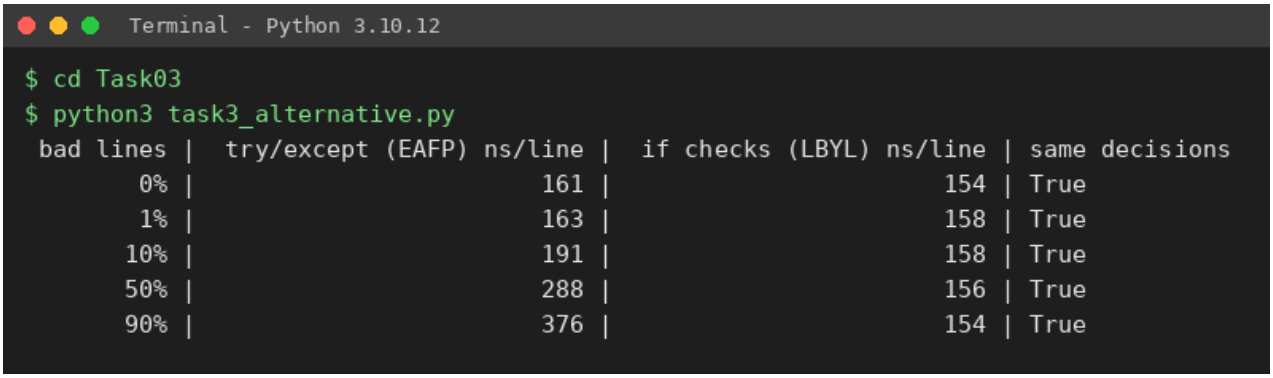
NameError would have crashed the program at once with a line number, and I would have fixed it in a minute.

8. Analysis

- The completed program reproduced the assignment's expected output exactly, so the two TODOs are correct.
- In the processor run: good.txt returned 2 valid lines and 2 bad lines (line 3 'expected 3 fields, found 2', line 4 "bad date '08-10-2026'"). The four failures gave a ProcessingError with the right cause kept: FileNotFoundError (twice), PermissionError and UnicodeDecodeError. The empty file gave a ProcessingError with no cause, as I predicted. 'file closed' appears for all six, so finally really runs every time. 'finished' appears only for the good file, so else runs only on success.
- The last line of that run shows raise ... from ...: the InvalidRecord for '31-31-2026' has the original ValueError as its cause, so the error message and the reason both survive.
- All of this matched my predictions.

9. Alternative Concept and Complexity Analysis

Alternative: check every line with if statements before using it ('look before you leap', LBYL) instead of raising and catching exceptions ('easier to ask forgiveness', EAFP). I wrote both checks (task3_alternative.py) and timed 300,000 lines with different shares of bad lines. Both made the same decision on every line.



Style	Good line	Bad line	Time for n lines	Space
try / except (EAFP)	just does the work, no extra test	raising and catching an exception costs extra time	O(n) in both cases, but the cost per line grows with the share of bad lines	O(1) extra
if checks first (LBYL)	pays for the checks on every line	the failed check costs the same as a passed one	O(n) with almost the same cost per line whatever the share of bad lines	O(1) extra

Table T3-2: complexity. Both styles are O(n) in time and O(1) in extra space; only the cost per line is different.

Reading my numbers: with 0% and 1% bad lines the two styles cost about the same per line (about 155 against 149 ns). As the share of bad lines grew, the exception style became slower per line (187 ns at 10%, 285 ns at 50%, 372 ns at 90%), while the if style stayed at about 150 ns. So when bad lines are rare, exceptions cost almost nothing, and the code is shorter and cannot forget a check; when bad lines are common, checking first is cheaper. In my attendance files bad lines are rare, so I kept the exception style and, more importantly, it lets me give each kind of failure its own name (InvalidRecord) and message.

10. Conclusion

Naming the failures first (Table T3-1) made the handlers easy: one custom exception for a damaged line, one for an unusable file, raise … from to keep the real cause, and finally to always close the file. The completed program matched the expected output exactly, and the misuse experiment showed how except Exception: pass turned a typing error into a believable empty result.

TASK 4 - FILE AND EXCEPTION FORENSICS (7 Marks)

1. Aim

To find out from the code why the library log kept only the last day's books, why the titles were stuck together, and why one day stopped with no saved message, to prove each defect with a test, to repair the program, and to compare the with statement with manual try/finally.

2. Predictions made before running

Symptom reported by the library	My prediction from reading the code	Line responsible
after a week only the last day's books are in the log	open('returns.log', 'w') empties the file every time save_returns() is called	log = open('returns.log', "w")
books are joined together in the log	log.write(b) writes the title but no newline, so the next title follows on the same line	log.write(b)
when write() fails the file stays open	log.close() is after the loop; an exception skips it, nothing closes the file	log.close()
one day stopped with an error and no message was saved	only FileNotFoundError is caught; any other exception (for example a bad character, UnicodeDecodeError) is not caught and the program stops; even the 'missing' case is only print()ed, never saved	except FileNotFoundError / print('missing')
(extra) an empty title at the end of every day	read().split('\n') leaves '' after the last newline, and that empty string is passed on	open(day_file).read().split ("\n")

Table T4-1: predictions before running anything. Test files: day1.txt (3 books), day2.txt (2 books), day3.txt (4 books), day4_damaged.txt (has bytes that are not UTF-8).

3. Program

Task04/task4_original.py is copied exactly from the assignment. Task04/task4_evidence.py is the test that runs it.

```
1  # Task 4 - the library program, copied exactly from the assignment.
2  def save_returns(books):
3      log = open("returns.log", "w")
4      for b in books:
5          log.write(b)
6      log.close()
7
8
9  def process(day_file):
10     try:
11         books = open(day_file).read().split("\n")
12         save_returns(books)
13     except FileNotFoundError:
14         print("missing")
```

4. Task 4.1 and 4.2 - Locate the Defects and Construct the Evidence (4 marks)

The test runs process() for three days and prints the log after each day (test A), makes write() fail halfway and checks whether the log file was closed (test B), feeds a damaged file (test C) and a missing file (test D).

```python
"""Task 4.2 - tests that demonstrate each defect of the original program."""
import os, sys
from task4_original import process, save_returns

def show_log(title):
    print(f"[{title}] returns.log contains: {open('returns.log', encoding='utf-8').read()!r}")

if os.path.exists("returns.log"):
    os.remove("returns.log")

print("TEST A - process() for three days (day1: 3 books, day2: 2 books, day3: 4 books)")
for d in ("day1.txt", "day2.txt", "day3.txt"):
    process(d)
    show_log("after " + d)
print("  -> defect 1: only day3's books remain (mode 'w' empties the file every call)")
print("  -> defect 2: titles are glued together, e.g. 'GodanPonniyin...' (no newline written)")
print("  -> defect 3: split('\\n') leaves an empty string '' at the end of every day's list")

print("\nTEST B - a write that fails halfway (the list has a number in it)")
try:
    save_returns(["Book A", 42, "Book C"])
except TypeError as err:
    tb = sys.exc_info()[2]
    handle = tb.tb_next.tb_frame.f_locals["log"]
    print("  TypeError:", err)
    print("  is the log file closed?", handle.closed, "<- log.close() never ran")
    handle.close()

print("\nTEST C - a day file with a damaged character (not UTF-8), program must not stop silently")
try:
    process("day4_damaged.txt")
    print("  process() finished")
except Exception as err:
    print("  process() stopped with", type(err).__name__, "- nothing was printed or saved by the program itself")
print("\nTEST D - a missing file")
process("day9.txt")
print("  -> only the word 'missing' is printed: which file? when? it is not saved anywhere")
```

```
Terminal - Python 3.10.12

$ cd Task04
$ rm -f returns.log errors.log; python3 task4_evidence.py
TEST A - process() for three days (day1: 3 books, day2: 2 books, day3: 4 books)
[after day1.txt] returns.log contains: 'Wings of FireMalgudi DaysGodan'
[after day2.txt] returns.log contains: 'Ponniyin SelvanGitanjali'
[after day3.txt] returns.log contains: 'The GuideTrain to PakistanDiscovery of IndiaSwami and
    Friends'
  -> defect 1: only day3's books remain (mode 'w' empties the file every call)
  -> defect 2: titles are glued together, e.g. 'GodanPonniyin...' (no newline written)
  -> defect 3: split('\n') leaves an empty string '' at the end of every day's list

TEST B - a write that fails halfway (the list has a number in it)
  TypeError: write() argument must be str, not int
  is the log file closed? False <- log.close() never ran

TEST C - a day file with a damaged character (not UTF-8), program must not stop silently
  process() stopped with UnicodeDecodeError - nothing was printed or saved by the program itself

TEST D - a missing file
missing
  -> only the word 'missing' is printed: which file? when? it is not saved anywhere
```

- Why earlier days are lost: mode 'w'. After day 1 the log held 'Wings of FireMalgudi DaysGodan'; after day 2 it held only day 2's books; after day 3 only day 3's. Every call to save_returns() starts with an empty file, so after a week only the last day is left. (Test A.)
- Why the books are joined: log.write(b) has no newline, so the log contains 'Wings of FireMalgudi DaysGodan' as one long line. (Test A.)
- What happens to the open file when write() fails: in test B the third item is a number, so write() raised TypeError. I looked at the file object from the traceback and handle.closed was False. log.close() was

never reached, so the file stays open (and what was written to its buffer may never reach the disk). The same happens to the day file, which open(day_file).read() never closes at all.

- Why one day stopped with no message: test C. day4_damaged.txt has bytes that are not valid UTF-8, so read() raised UnicodeDecodeError. The program only catches FileNotFoundError, so it stopped and the program itself saved or printed nothing. Test D shows that even the handled case only prints the single word 'missing' on the screen: it does not say which file, and it is not written to any file.
- Extra defect: the empty string '' at the end of every day's list, so every day writes one empty title.

5. Task 4.3 - Repair the Program (2 marks)

File: Task04/task4_repaired.py. Append mode 'a', a newline after each book, with statements for both files, only real titles are kept (empty ones are dropped), FileNotFoundError and the other file errors are caught by name, and every error is written with the time, file name and error type to errors.log.

```python
"""Task 4.3 - repaired library program: append mode, newline after each book,
with statements, and an error log that is saved to a file."""
from datetime import datetime

LOG = "returns.log"
ERRORS = "errors.log"


def log_error(day_file, err):
    with open(ERRORS, "a", encoding="utf-8") as f:
        f.write(f"{datetime.now():%Y-%m-%d %H:%M:%S} {day_file}: {type(err).__name__}: {err}\n")


def save_returns(books):
    with open(LOG, "a", encoding="utf-8") as log:      # "a" keeps earlier days
        for b in books:
            log.write(b + "\n")                          # one title per line


def process(day_file):
    try:
        with open(day_file, encoding="utf-8") as f:
            books = [line.strip() for line in f if line.strip()]   # no empty titles
        save_returns(books)
    except FileNotFoundError as err:
        log_error(day_file, err)
        print("missing:", day_file)
    except (UnicodeDecodeError, PermissionError, OSError) as err:
        log_error(day_file, err)
        print("could not process", day_file, "-", type(err).__name__)


if __name__ == "__main__":
    import os
    for f in (LOG, ERRORS):
        if os.path.exists(f):
            os.remove(f)
    for d in ("day1.txt", "day2.txt", "day3.txt", "day4_damaged.txt", "day9.txt"):
        process(d)
    print("--- returns.log"); print(open(LOG, encoding="utf-8").read())
    print("--- errors.log"); print(open(ERRORS, encoding="utf-8").read())
```

```
Terminal - Python 3.10.12
$ cd Task04
$ python3 task4_repaired.py
could not process day4_damaged.txt - UnicodeDecodeError
missing: day9.txt
--- returns.log
Wings of Fire
Malgudi Days
Godan
Ponniyin Selvan
Gitanjali
The Guide
Train to Pakistan
Discovery of India
Swami and Friends

--- errors.log
2026-11-12 19:39:41 day4_damaged.txt: UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff in
    position 5: invalid start byte
2026-11-12 19:39:41 day9.txt: FileNotFoundError: [Errno 2] No such file or directory: 'day9.txt'
```

The repaired log keeps all 9 books from three days, one per line, in order, and the program did not stop at the damaged file or the missing file: each got one line in errors.log with the exact error type.

6. Task 4.4 - Prevent Recurrence (1 mark)

- Rule 1: always open files with a with statement, so the file is closed when the block ends, even if an exception happens inside (that would have fixed the open-file defect and the unclosed day file).
- Rule 2: choose the file mode on purpose and write one complete record at a time. Use 'a' for a log that grows day after day and 'w' only when I really mean to replace the file; end every record with '\n'. (That would have fixed the lost days and the joined titles.)

A third rule I would add on my own: catch exceptions by name and write them to an error log, never just print one word.

7. Analysis

- All five of my predictions in Table T4-1 were confirmed by a real run, so the defects are located, not guessed.
- The three visible symptoms came from two lines of the file-opening code: the 'w' mode and the missing newline. The invisible one (no message) came from the except clause that was too narrow and from print() being the only reporting.
- The repair is small, but the effect is large: with 'a' and '\n' the log now holds every day's books, and the error log now answers 'which file, when, what kind of error'.

8. Alternative Concept and Complexity Analysis

Alternative: open and close the files manually inside try/finally instead of using with. I wrote three versions of the same job (read a day file, append to the log) with a deliberate error halfway: the original style, manual try/finally, and with. A small wrapper around open() counted how many files were still open afterwards (task4_alternative.py).

```
Terminal - Python 3.10.12
$ cd Task04
$ python3 task4_alternative.py
version                 handles open after an error    handles open after success
original (no close)                                2                             1
manual try/finally                                 0                             0
with statement                                     0                             0

Time to process a 50,000-book day file (best of 5):
  manual try/finally      4.79 ms
  with statement          4.76 ms
```

Version	Files still open after an error (measured)	Time for n lines	Extra space
original (no close)	2 (the day file and the log)	O(n)	O(n) for the whole list of books
manual try/finally	0	O(n)	O(n) for the list (same as the others)
with statement	0	O(n)	O(n) for the list (same as the others)

Table T4-2: complexity and open files. Time for a 50,000-book day file: about 4.8 ms for both manual try/finally and with.

The time and space do not change: both correct versions read n lines once, so O(n) time, and they have the same space. Neither is faster; the measured times are the same (4.80 ms and 4.79 ms). What changes is how many files can stay open if an error happens: the original left 2 open, and the two correct versions left 0. The difference is in who has to remember: with try/finally I have to write the finally and the close() for each file myself, so with two files it is easy to forget one or to put the second open() inside the wrong try; the with statement does the closing by itself. With the manual version, a forgotten finally means up to 2 files remain open, the same as the original. The with version is also shorter, because it needs no finally block.

9. Conclusion

The three symptoms had two causes in the code, mode 'w' and no newline, and the lost message had a third, an except clause that was too narrow. Tests A to D proved each one. The repaired program uses append mode, newlines, with statements and an error log, and with statement versus manual try/finally has the same O(n) cost but leaves 0 files open against 2 for the original.

TASK 5 - THE SAME UTILITY ORGANISED FOUR WAYS (7 Marks)

1. Aim

To organise the same attendance utility (reading files, computing statistics, writing reports) as (a) one script, (b) one module imported by a main script, (c) three modules in one folder and (d) a package attendance/ with __init__.py and reader.py, stats.py, report.py, then to add a second report to each and count how many files had to change, and to compare the four.

2. Predictions made before running

- All four organisations should print identical output on the same daily files, because the functions are the same; only the place where they live changes.
- Adding the second report (students absent for three consecutive days) needs a new function in the statistics group, a new function in the report group and a call in the main part. So I predicted: (a) 1 file changes, (b) 2 files (module + main), (c) 3 files (stats.py, report.py and the main script), (d) 3 files (stats.py, report.py and the main script). The reader.py and __init__.py should not change.
- Importing the package for the first time should cost more than importing one script, because Python has to find, read and run four files instead of one, but a second import should cost almost nothing (sys.modules).

3. Task 5.1 - Four Organisations (3 marks)

All four read the same 8 daily files (Task05/daily, the same files as Task 2) and write report_percentages.txt. Folder layout:

```
Task05/
  daily/                      the 8 daily files
  org_a_one_script/   attendance_all.py          (reader + stats + report + main in one
file)
  org_b_one_module/   attendance_utils.py  run_b.py
  org_c_three_modules/ reader.py  stats.py  report.py  run_c.py
  org_d_package/      attendance/__init__.py  attendance/reader.py
                      attendance/stats.py  attendance/report.py  run_d.py
```

(a) One script. Everything is in attendance_all.py. The main part is protected by if __name__ == "__main__": so that importing the file (for a test) does not run the program.

```python
"""Organisation (a): everything in one script."""
from datetime import date
from pathlib import Path

def read_files(folder):
    """Read every *.txt in the folder. Returns (records, errors)."""
    records, errors = [], []
    for path in sorted(Path(folder).glob("*.txt")):
        try:
            with open(path, encoding="utf-8") as f:
                for number, line in enumerate(f, start=1):
                    line = line.strip()
                    if not line:
                        continue
                    parts = line.split(",")
                    if len(parts) != 3:
                        errors.append((path.name, number, "expected 3 fields"))
                        continue
                    reg, day, status = parts
                    try:
                        date.fromisoformat(day)
                    except ValueError:
                        errors.append((path.name, number, "bad date"))
                        continue
                    if status not in ("P", "A"):
                        errors.append((path.name, number, "bad status"))
                        continue
                    records.append((reg, day, status))
        except (OSError, UnicodeDecodeError) as err:
            errors.append((path.name, 0, "unreadable: " + type(err).__name__))
    return records, errors


def percentages(records):
    """records -> {register: (present, total, percentage)}"""
    counts = {}
    for reg, day, status in records:
        c = counts.setdefault(reg, [0, 0])
        c[1] += 1
        c[0] += status == "P"
    return {reg: (p, t, 100 * p / t) for reg, (p, t) in counts.items()}


def below(pcts, limit=75):
    return sorted(reg for reg, (p, t, x) in pcts.items() if x < limit)


# REQ2-BEGIN  second report: students absent for three consecutive days
def absent_streaks(records, run=3):
    days = {}
    for reg, day, status in records:
        if status == "A":
```

```
            days.setdefault(reg, []).append(date.fromisoformat(day).toordinal())
    found = []
    for reg, ordinals in days.items():
        ordinals.sort()
        streak = best = 1
        for a, b in zip(ordinals, ordinals[1:]):
            streak = streak + 1 if b - a == 1 else 1
            best = max(best, streak)
        if best >= run:
            found.append(reg)
    return sorted(found)
# REQ2-END


def write_report(pcts, low, path):
    with open(path, "w", encoding="utf-8") as out:
        out.write("Register   Present  Total  Percentage\n")
        for reg in sorted(pcts):
            p, t, x = pcts[reg]
            out.write(f"{reg}  {p:7d}  {t:5d}  {x:9.1f}%\n")
        out.write("Below 75%: " + (", ".join(low) if low else "none") + "\n")


# REQ2-BEGIN  second report file
def write_absent_report(regs, path):
    with open(path, "w", encoding="utf-8") as out:
        out.write("Absent for three consecutive days:\n")
        for reg in regs:
            out.write(f"  {reg}\n")
        if not regs:
            out.write("  none\n")
# REQ2-END

def main():
    folder = Path(__file__).resolve().parent.parent / "daily"
    here = Path(__file__).resolve().parent
    records, errors = read_files(folder)
    pcts = percentages(records)
    low = below(pcts)
    write_report(pcts, low, here / "report_percentages.txt")
    print(open(here / "report_percentages.txt", encoding="utf-8").read())
    print("damaged lines skipped:", len(errors))
    # REQ2-BEGIN  second report (Task 5.2)
    streaks = absent_streaks(records)
    write_absent_report(streaks, here / "report_absent3.txt")
    print(open(here / "report_absent3.txt", encoding="utf-8").read())
    # REQ2-END


if __name__ == "__main__":
    main()
```

(b) One module imported by a main script. attendance_utils.py holds the same functions as (a) (it is the same text, without the main part). run_b.py imports it as au and calls au.read_files(...) and so on.

```python
"""Main script of organisation (b)."""
from pathlib import Path
import attendance_utils as au

def main():
    folder = Path(__file__).resolve().parent.parent / "daily"
    here = Path(__file__).resolve().parent
    records, errors = au.read_files(folder)
    pcts = au.percentages(records)
    low = au.below(pcts)
    au.write_report(pcts, low, here / "report_percentages.txt")
    print(open(here / "report_percentages.txt", encoding="utf-8").read())
    print("damaged lines skipped:", len(errors))
    # REQ2-BEGIN  second report (Task 5.2)
    streaks = au.absent_streaks(records)
    au.write_absent_report(streaks, here / "report_absent3.txt")
    print(open(here / "report_absent3.txt", encoding="utf-8").read())
    # REQ2-END


if __name__ == "__main__":
    main()
```

(c) Three modules in one folder. reader.py, stats.py and report.py each hold one group. run_c.py does import reader, stats, report. Because run_c.py is in the same folder, Python finds them.

```python
"""Organisation (c): statistics."""
from datetime import date

def percentages(records):
    """records -> {register: (present, total, percentage)}"""
    counts = {}
    for reg, day, status in records:
        c = counts.setdefault(reg, [0, 0])
        c[1] += 1
        c[0] += status == "P"
    return {reg: (p, t, 100 * p / t) for reg, (p, t) in counts.items()}


def below(pcts, limit=75):
    return sorted(reg for reg, (p, t, x) in pcts.items() if x < limit)


# REQ2-BEGIN  second report: students absent for three consecutive days
def absent_streaks(records, run=3):
    days = {}
    for reg, day, status in records:
        if status == "A":
            days.setdefault(reg, []).append(date.fromisoformat(day).toordinal())
    found = []
    for reg, ordinals in days.items():
        ordinals.sort()
        streak = best = 1
        for a, b in zip(ordinals, ordinals[1:]):
            streak = streak + 1 if b - a == 1 else 1
            best = max(best, streak)
        if best >= run:
            found.append(reg)
    return sorted(found)
# REQ2-END
```

```
"""Organisation (c): report files."""

def write_report(pcts, low, path):
    with open(path, "w", encoding="utf-8") as out:
        out.write("Register   Present  Total  Percentage\n")
        for reg in sorted(pcts):
            p, t, x = pcts[reg]
            out.write(f"{reg}  {p:7d}  {t:5d}  {x:9.1f}%\n")
        out.write("Below 75%: " + (", ".join(low) if low else "none") + "\n")


# REQ2-BEGIN  second report file
def write_absent_report(regs, path):
    with open(path, "w", encoding="utf-8") as out:
        out.write("Absent for three consecutive days:\n")
        for reg in regs:
            out.write(f"  {reg}\n")
        if not regs:
            out.write("  none\n")
# REQ2-END
```

```
"""Main script of organisation (c)."""
from pathlib import Path
import reader, stats, report

def main():
    folder = Path(__file__).resolve().parent.parent / "daily"
    here = Path(__file__).resolve().parent
    records, errors = reader.read_files(folder)
    pcts = stats.percentages(records)
    low = stats.below(pcts)
    report.write_report(pcts, low, here / "report_percentages.txt")
    print(open(here / "report_percentages.txt", encoding="utf-8").read())
    print("damaged lines skipped:", len(errors))
    # REQ2-BEGIN  second report (Task 5.2)
    streaks = stats.absent_streaks(records)
    report.write_absent_report(streaks, here / "report_absent3.txt")
    print(open(here / "report_absent3.txt", encoding="utf-8").read())
    # REQ2-END


if __name__ == "__main__":
    main()
```

(d) A package attendance/. The folder attendance/ has __init__.py (it makes the folder a package, and lists the sub-modules in __all__) and reader.py, stats.py, report.py (same functions as in (c)). run_d.py uses from attendance import reader, stats, report.

```
"""Organisation (d): package attendance = reader + stats + report."""
__all__ = ["reader", "stats", "report"]
__version__ = "1.0"
```

```python
"""Main script of organisation (d)."""
from pathlib import Path
from attendance import reader, stats, report

def main():
    folder = Path(__file__).resolve().parent.parent / "daily"
    here = Path(__file__).resolve().parent
    records, errors = reader.read_files(folder)
    pcts = stats.percentages(records)
    low = stats.below(pcts)
    report.write_report(pcts, low, here / "report_percentages.txt")
    print(open(here / "report_percentages.txt", encoding="utf-8").read())
    print("damaged lines skipped:", len(errors))
    # REQ2-BEGIN  second report (Task 5.2)
    streaks = stats.absent_streaks(records)
    report.write_absent_report(streaks, here / "report_absent3.txt")
    print(open(here / "report_absent3.txt", encoding="utf-8").read())
    # REQ2-END


if __name__ == "__main__":
    main()
```

4. Output

The real output of the four programs (the first report is the same in all four; the second report is explained in 5.2):

```
Terminal - Python 3.10.12

$ cd Task05
$ python3 org_a_one_script/attendance_all.py
Register   Present  Total  Percentage
192311001        8      8      100.0%
192311002        7      7      100.0%
192311003        4      8       50.0%
192311004        6      7       85.7%
192311005        4      8       50.0%
192311006        6      7       85.7%
Below 75%: 192311003, 192311005

damaged lines skipped: 3
Absent for three consecutive days:
  192311003
```

```
Terminal - Python 3.10.12
$ cd Task05
$ python3 org_b_one_module/run_b.py
Register   Present  Total  Percentage
192311001        8      8      100.0%
192311002        7      7      100.0%
192311003        4      8       50.0%
192311004        6      7       85.7%
192311005        4      8       50.0%
192311006        6      7       85.7%
Below 75%: 192311003, 192311005

damaged lines skipped: 3
Absent for three consecutive days:
  192311003
```

```
Terminal - Python 3.10.12
$ cd Task05
$ python3 org_c_three_modules/run_c.py
Register   Present  Total  Percentage
192311001        8      8      100.0%
192311002        7      7      100.0%
192311003        4      8       50.0%
192311004        6      7       85.7%
192311005        4      8       50.0%
192311006        6      7       85.7%
Below 75%: 192311003, 192311005

damaged lines skipped: 3
Absent for three consecutive days:
  192311003
```

```
Terminal - Python 3.10.12
$ cd Task05
$ python3 org_d_package/run_d.py
Register   Present  Total  Percentage
192311001        8      8      100.0%
192311002        7      7      100.0%
192311003        4      8       50.0%
192311004        6      7       85.7%
192311005        4      8       50.0%
192311006        6      7       85.7%
Below 75%: 192311003, 192311005

damaged lines skipped: 3
Absent for three consecutive days:
  192311003
```

I compared the four outputs with each other: they are identical, 4 of 4, as predicted.

5. Task 5.2 - Change the Requirement (2 marks)

The department wants a second report: a list of students absent for three consecutive days. In every organisation the new code is marked between the comments # REQ2-BEGIN and # REQ2-END: absent_streaks() in the statistics group, write_absent_report() in the report group, and three lines in the main part. To count the changes honestly, task5_change_effort.py makes a copy of every organisation without the marked blocks (the first version), runs it to prove it works and prints only the first report, and then counts the files and lines that contain the marked blocks.

```
Terminal - Python 3.10.12
$ cd Task05
$ python3 task5_change_effort.py
organisation       files  files changed  lines added   first version output
a one script           1              1           31   runs, first report only
b one module           2              2           31   runs, first report only
c three modules        4              3           31   runs, first report only
d package              5              3           31   runs, first report only
```

Organisation	Python files	Files that had to change	Which
(a) one script	1	1	attendance_all.py
(b) one module + main	2	2	attendance_utils.py, run_b.py
(c) three modules	4	3	stats.py, report.py, run_c.py (reader.py unchanged)
(d) package	5	3	attendance/stats.py, attendance/report.py, run_d.py (reader.py and __init__.py unchanged)

Table T5-1: files changed. This matches my prediction exactly. The same 31 lines were added in every case.

The result for the new report in the program output (the second report is the last part of each run above): only 192311003 was absent for three consecutive days (days 2, 3 and 4). Another student, 192311005, was absent four days but not three in a row (1, 2, 6, 7), so it is correctly not listed.

6. Task 5.3 - Compare the Organisations (2 marks)

	(a) one script	(b) one module	(c) three modules	(d) package
Readability	good for 100 lines; gets long and hard to scroll as it grows (103 lines already)	main script is short and clear; the module is still 84 lines	each file is short (20 to 34 lines) and has one job	same as (c), and the folder name tells what belongs together
Reuse	poor: to use one function in another project I must copy it or import the whole script	good: import the module in any project	good: import only what I need, for example just stats	best: one import name, can be installed or copied as a unit
Testing	hard to test one function without running or importing everything	can import and test functions	can test each module on its own	same as (c); tests can import attendance.stats directly
Naming conflicts	none between files, but all names share one space	names are inside the module (au.read_files)	top-level names reader, stats, report can clash with another stats.py on the path	clashes are avoided: the names are attendance.stats, attendance.reader ...

Table T5-2: comparison

Difference between the three import styles, shown by a real run of task5_import_styles.py:

```
Terminal - Python 3.10.12
$ cd Task05
$ python3 task5_import_styles.py
1) import attendance.stats
   names created : ['attendance']
   how I call it : attendance.stats.percentages(...)  -> attendance.stats
2) from attendance import stats
   names created : ['attendance', 'stats']
   how I call it : stats.percentages(...)
3) from attendance.stats import percentages
   names created : ['attendance', 'percentages', 'stats']
   how I call it : percentages(...)   (the module name is not visible any more)
   modules loaded: ['attendance', 'attendance.stats']

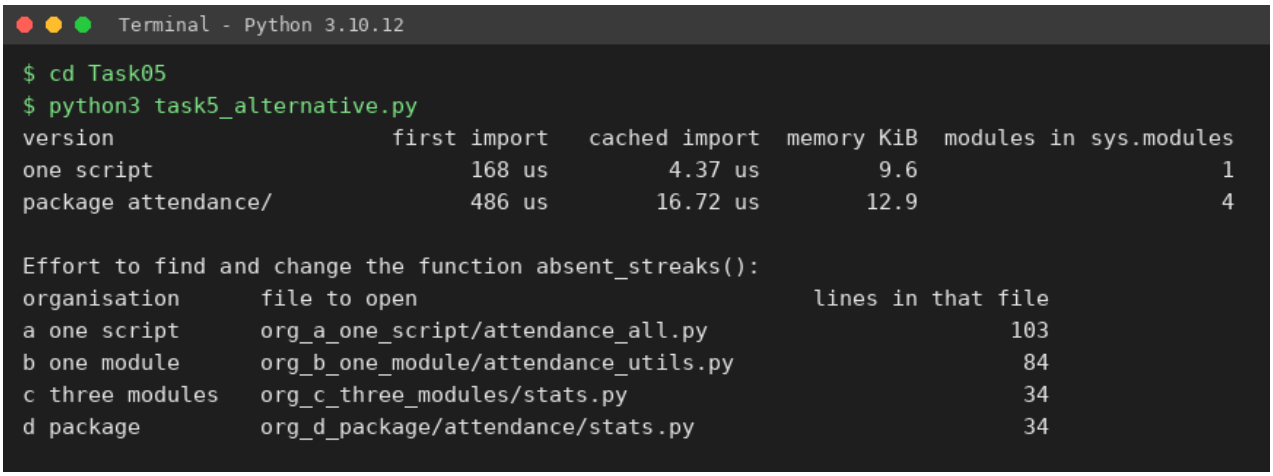
Naming conflict A: a later definition silently replaces an imported name
   percentages([]) now -> MY OWN FUNCTION (the imported one is gone from this file)
   stats.percentages([]) still -> {} (module style is safe)

Naming conflict B: two folders both containing a module called stats.py
   import stats -> <other folder>/stats.py
   attendance.stats is still attendance.stats because the package name is part of it
```

- import attendance.stats loads the package and the sub-module, but the only new name in my file is attendance. I must write the full path every time: attendance.stats.percentages(...).
- from attendance import stats makes the name stats point to the sub-module, so I write stats.percentages(...). It is short and still shows which module a function comes from.
- from attendance.stats import percentages brings only the function. I can write percentages(...), but the module name is no longer visible and, as the run shows (conflict A), if my own file later defines percentages, it silently replaces the imported one. That is the usual source of naming conflicts. In all three styles Python loads attendance.stats once; the list of loaded modules shows attendance and attendance.stats.
- Conflict B in the run shows the risk of option (c): when two folders both have a stats.py, import stats gives whichever folder comes first on sys.path. In the package the module is always attendance.stats, so it cannot be confused.

7. Alternative Concept and Complexity Analysis

Alternative: keep all functions in one script instead of a package. I measured the one-time cost of importing the one script and the package (with its three sub-modules) with time.perf_counter, the memory with tracemalloc, and the effort of finding and changing absent_streaks() as the number of lines of the file I have to open (task5_alternative.py).



Version	First import	Import again (cached)	Memory for the import	Modules in sys.modules	Lines of the file to open to change absent_streaks()
one script	168 us	4.37 us	9.6 KiB	1	103
package attendance/	486 us	16.72 us	12.9 KiB	4	34 (stats.py)

Table T5-3: measured (best of 5). Complexity: first import O(m) for m modules (each file is found, read and run once); every later import is an O(1) lookup in sys.modules.

The package costs more only once: the first import took 486 us against 168 us for the single script, because Python finds, reads and runs four files instead of one (the package also uses a little more memory, 12.9 against 9.6 KiB). After that, a repeated import is only a dictionary lookup in sys.modules (16.72 us against 4.37 us), so a program that imports the package in ten places still loads it once. These costs are microseconds, so they do not matter for this program. What does matter is the effort of changing code: in the package I open a 34-line stats.py to change a statistics function, but in the single script I open and

search a 103-line file, and every unrelated function is in front of me. The cost of the package is more files and a little more thinking about imports; the benefit is that the code is easier to find, reuse and test, and a change touches fewer lines.

8. Conclusion

All four organisations gave identical output. Adding the second report changed 1, 2, 3 and 3 files, exactly as I predicted, and in (c) and (d) the reader never needed to change. The package costs a few hundred microseconds on the first import and nothing afterwards, and it avoids naming conflicts, so I would choose the package for anything bigger than a short script.

TASK 6 - DESIGN CHALLENGE: COLLEGE FEST REGISTRATION DESK (13 Marks)

1. Aim

To design, build and demonstrate a program (a package called festdesk) that reads registrations from five department files, removes duplicates, validates e-mail addresses with string methods, writes a clean master list, one participant file per event, a summary report and an error log, and never stops because of a bad line or a missing file. I was not given the design, so the five stages below are my own.

2. Stage 1 - Interpret the Problem (2 marks)

Inputs: five text files CSE.txt, ECE.txt, EEE.txt, MECH.txt, CIVIL.txt. Each line: register number, name, department, e-mail, events separated by semicolons, for example 192311001,arun kumar,CSE,arun.k@college.edu,Dance;Quiz. Outputs: master_list.csv, events/<event>.txt (one per event), summary_report.txt and errors.log. Rules and requirements as testable statements (each has a test in Stage 5):

No.	Requirement (testable statement)	Test case that checks it
R1	All five department files are read when they exist.	clean: 15 lines read, 15 accepted
R2	A line is valid only if it has exactly 5 comma-separated fields, a register number of 6 to 15 letters/digits, a non-empty name and at least one event.	damaged
R3	An e-mail is valid only if it has exactly one @, something before the @, a dot in the part after the @, no dot at the start, at the end or twice in a row there, a domain ending of at least 2 letters, and no spaces.	bad_email
R4	A register number appears at most once in the master list. The first registration is kept and every later one (same file or another department) is logged as DUPLICATE.	duplicate
R5	A missing or unreadable department file is logged as MISSING and the other files are still processed.	missing
R6	A bad line never stops the program: the exit code is 0 and the other lines are still processed.	all six runs
R7	master_list.csv has exactly one row per accepted student.	clean: 15 rows
R8	For every event there is a file listing exactly the students who chose that event, with the count in its first line.	clean: quiz.txt
R9	errors.log has one line for each rejected line or missing file, with the kind, the file name and the line number.	all problem cases
R10	summary_report.txt gives the number of files expected and missing, lines read, accepted students, rejected lines by kind, students per department and participants per event.	all_problems
R11	Blank lines are ignored and are not errors.	damaged

Table T6-1: requirements

3. Stage 2 - Propose the Processing Flow (2 marks)

The flow from the input files to the output files is in the picture. Validation happens straight after a line is read; the duplicate check comes only after validation passes, so a damaged line is never put in the 'seen'

set. Every rejection goes to the error list with its file and line number, then the loop continues. The output files are written once at the end, when all the lists are complete.

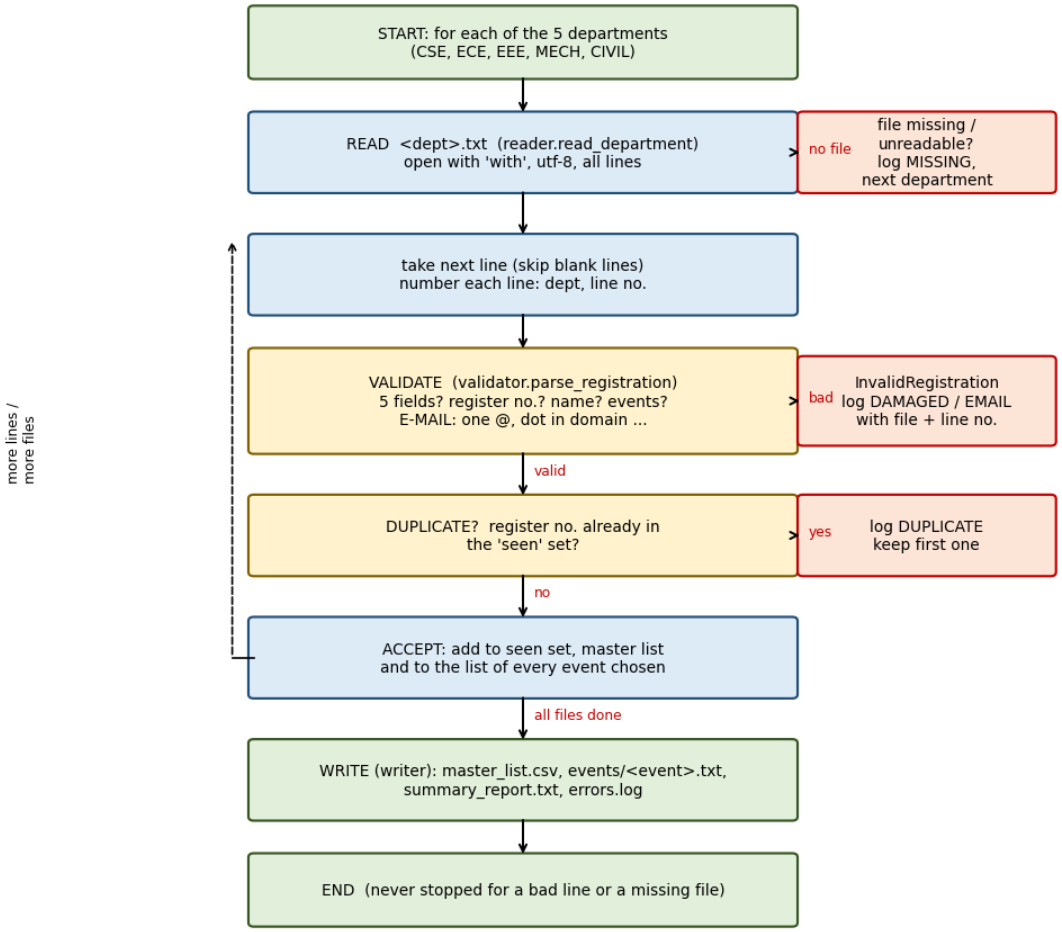


Figure T6-1: processing flow (red boxes = where errors are logged)

4. Stage 3 - Identify the Critical Design Decisions (2 marks)

Decision	What I chose	Why (and what else I considered)
How duplicates are detected	A set called seen holds the register numbers (upper-case) accepted so far. For each valid line: if reg in seen it is a duplicate, else seen.add(reg).	A set looks up in O(1) on average. Searching a list of earlier registrations costs O(n) per line, so O(n^2) for the whole run (measured below). The key is the register number only, not the name, because names can be spelled differently. The check covers all five files, because one student may register in two departments.
Which duplicate wins	The first one is kept; later ones are logged and ignored.	Simple and predictable. Merging the events of the two registrations would also be possible, but the committee can decide that from the log, and I would not invent a student's choices.
How e-mails are validated (string methods)	count('@') == 1; split('@') into local and domain; local != ''; '.' in domain; not startswith('.'), not endswith('.'), '..' not in domain; rsplit('.', 1)[1].isalpha() and at least 2 letters; ' ' not in	These are the string methods the course covers. It is not a full check of every rule for e-mail addresses, but it catches the common mistakes (no @, two @, no dot, spaces). Only sending a

	email. Then lower() for the master list.	message could prove that an address exists, and that is outside this assignment.
Which exceptions are handled where	reader.read_department(): FileNotFoundError, PermissionError, UnicodeDecodeError, returned as a 'problem' so the loop goes to the next department. validator.parse_registration(): raises my InvalidRegistration (kind DAMAGED or EMAIL). desk.run(): except InvalidRegistration around each line. run_fest.py: OSError only for writing the output folder.	Each exception is handled at the first place that has enough information to continue: the reader knows which file, run() knows the line number. There is no 'except Exception' anywhere, because of what my Task 3 misuse experiment showed: a bug of mine would be hidden. If the output folder itself cannot be written, the program stops with a clear message, because nothing useful can be done.

Table T6-2: critical design decisions

5. Stage 4 - Design the Package (4 marks)

Module	Functions / classes	Job
festdesk/__init__.py	run, InvalidRegistration (re-exported), __version__	makes the folder a package; run_fest.py uses: from festdesk import run
festdesk/errors.py	class InvalidRegistration(kind, reason)	my own exception for one rejected line
festdesk/validator.py	check_email(email), parse_registration(line)	all checking of one line; no file access at all
festdesk/reader.py	read_department(path), read_all(input_dir, departments)	all reading; turns file problems into a 'problem' text
festdesk/writer.py	write_master, write_events, write_errors, write_summary, event_file_name	all writing of the four kinds of output
festdesk/desk.py	run(input_dir, output_dir, departments)	the whole flow; calls the other modules and holds the seen set
run_fest.py	(main script)	command line: python3 run_fest.py <input> <output>

Table T6-3: package structure

Pseudocode for reading, validating and writing (SIMATS Engineering algorithm style):

```
ALGORITHM ReadDepartmentFile
INPUT  : path of a department file
OUTPUT : list of (lineNo, text) and a problem message (or none)
BEGIN
 1.  TRY
 2.      OPEN path for reading (utf-8)
 3.      lines <- every line of the file with its number
 4.      CLOSE path
 5.      RETURN lines, none
 6.  ON FileNotFoundError  RETURN empty list, 'department file is missing'
 7.  ON PermissionError    RETURN empty list, 'no permission to read the file'
 8.  ON UnicodeDecodeError RETURN empty list, 'file is not UTF-8 text'
 9.  END TRY
END ALGORITHM

ALGORITHM ValidateRegistration
INPUT  : one text line
OUTPUT : a student record, or InvalidRegistration is raised
BEGIN
 1.  fields <- split line at ',' and strip each field
 2.  IF number of fields <> 5 THEN RAISE InvalidRegistration(DAMAGED)
 3.  IF register is not 6-15 letters/digits THEN RAISE InvalidRegistration(DAMAGED)
 4.  IF name is empty THEN RAISE InvalidRegistration(DAMAGED)
 5.  events <- split events at ';', strip, title-case, remove blanks and repeats
 6.  IF events is empty THEN RAISE InvalidRegistration(DAMAGED)
 7.  problem <- CheckEmail(email)
 8.  IF problem exists THEN RAISE InvalidRegistration(EMAIL, problem)
```

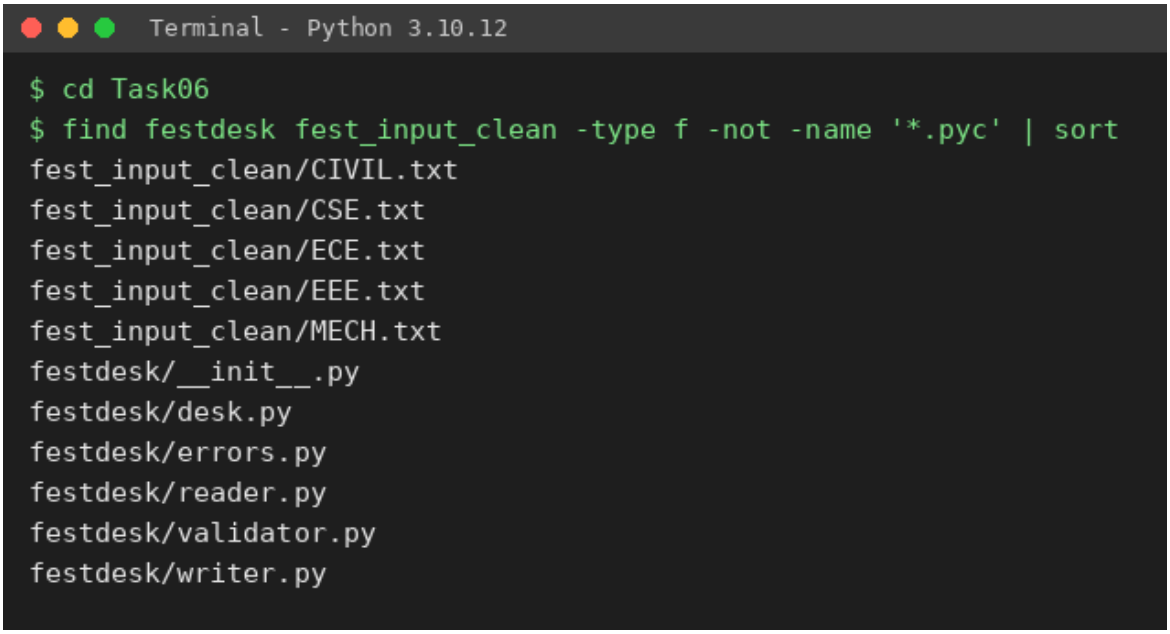
```
 9.  RETURN record(register, name, department, lower-case email, events)
END ALGORITHM

ALGORITHM WriteOutputs
INPUT  : accepted students, events dictionary, problems, statistics, output folder
OUTPUT : master_list.csv, events/<event>.txt, errors.log, summary_report.txt
BEGIN
 1.  CREATE output folder if it does not exist
 2.  WRITE a header and one row per accepted student to master_list.csv
 3.  FOR EACH event IN events DO
 4.      WRITE '<event> - <count> participants' and the sorted participants
 5.          to events/<event>.txt
 6.  END FOR
 7.  FOR EACH problem IN problems DO WRITE kind, place and text TO errors.log
 8.  WRITE the counts and the per-department and per-event numbers TO summary_report.txt
END ALGORITHM

ALGORITHM RunFestDesk
INPUT  : input folder, output folder
BEGIN
 1.  seen <- empty set;  students <- empty list;  problems <- empty list
 2.  FOR EACH department IN (CSE, ECE, EEE, MECH, CIVIL) DO
 3.      lines, problem <- ReadDepartmentFile(department file)
 4.      IF problem exists THEN ADD (file, MISSING, problem) TO problems; NEXT department
 5.      FOR EACH (lineNo, text) IN lines DO
 6.          IF text is blank THEN NEXT line
 7.          TRY  record <- ValidateRegistration(text)
 8.          ON InvalidRegistration  ADD to problems; NEXT line
 9.          IF record.register IN seen THEN ADD DUPLICATE to problems; NEXT line
10.          ADD record.register TO seen; ADD record TO students
11.          FOR EACH event IN record.events DO add (register, name) to events[event]
12.      END FOR
13.  END FOR
14.  WriteOutputs(...)
END ALGORITHM
```

6. Stage 5 - Implement and Demonstrate (3 marks)

Folder: Task06/. The package is festdesk/ and the sample input folders are fest_input_clean, fest_input_duplicate, fest_input_bad_email, fest_input_damaged, fest_input_missing and fest_input_all_problems (made by make_fest_samples.py). The folders and files created:



The package code:

```
"""festdesk - college fest registration desk package.

errors     - the InvalidRegistration exception
validator  - checks one line (fields, register number, e-mail, events)
reader     - reads the department files without ever stopping on a missing file
writer     - writes the master list, event lists, summary report and error log
desk       - run(): the whole flow, calls the others
"""
from .desk import run
from .errors import InvalidRegistration

__all__ = ["run", "InvalidRegistration"]
__version__ = "1.0"
```

```
"""Exceptions of the fest desk."""


class InvalidRegistration(Exception):
    """One registration line cannot be accepted.

    kind is 'DAMAGED' (wrong shape) or 'EMAIL' (bad e-mail address); the reason is a sentence."""

    def __init__(self, kind, reason):
        super().__init__(reason)
        self.kind = kind
        self.reason = reason
```

```
"""Validation of one registration line using string methods only."""
from .errors import InvalidRegistration


def check_email(email):
    """Return None if the e-mail looks valid, otherwise the reason as text."""
    if email == "" or " " in email:
        return "e-mail is empty or contains a space"
    if email.count("@") != 1:
        return "e-mail must contain exactly one @"
    local, domain = email.split("@")
    if local == "":
        return "nothing before the @"
    if "." not in domain:
        return "domain has no dot"
    if domain.startswith(".") or domain.endswith(".") or ".." in domain:
        return "misplaced dot in the domain"
    ending = domain.rsplit(".", 1)[1]
    if len(ending) < 2 or not ending.isalpha():
        return "domain ending is not letters"
    return None


def parse_registration(line):
    """'reg,name,dept,email,event1;event2' -> dict. Raises InvalidRegistration."""
    parts = [p.strip() for p in line.strip().split(",")]
    if len(parts) != 5:
        raise InvalidRegistration("DAMAGED", f"expected 5 fields, found {len(parts)}")
    reg, name, dept, email, events_text = parts
    reg = reg.upper()
    if not reg.isalnum() or not 6 <= len(reg) <= 15:
        raise InvalidRegistration("DAMAGED", f"register number {reg!r} is not 6-15 letters/digits")
    if name == "":
        raise InvalidRegistration("DAMAGED", "name is empty")
    events = []
    for e in events_text.split(";"):
        e = e.strip().title()
        if e and e not in events:
            events.append(e)
    if not events:
        raise InvalidRegistration("DAMAGED", "no event chosen")
    problem = check_email(email)
    if problem:
        raise InvalidRegistration("EMAIL", problem)
    return {"reg": reg, "name": name.title(), "dept": dept.upper(),
            "email": email.lower(), "events": events}
```

```python
"""Reading the department files. A missing or unreadable file is reported, never fatal."""
from pathlib import Path


def read_department(path):
    """Return (lines, problem). lines = [(line number, text)], problem = None or a sentence."""
    try:
        with open(path, encoding="utf-8") as f:
            return [(n, line.rstrip("\n")) for n, line in enumerate(f, start=1)], None
    except FileNotFoundError:
        return [], "department file is missing"
    except PermissionError:
        return [], "no permission to read the file"
    except UnicodeDecodeError:
        return [], "file is not UTF-8 text"


def read_all(input_dir, departments):
    """Yield (department, lines, problem) for each expected department."""
    for dept in departments:
        lines, problem = read_department(Path(input_dir) / f"{dept}.txt")
        yield dept, lines, problem
```

```python
"""Writing the output files."""
import csv
from pathlib import Path


def event_file_name(event):
    return event.lower().replace(" ", "_") + ".txt"


def write_master(path, students):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["register", "name", "department", "email", "events"])
        for s in students:
            w.writerow([s["reg"], s["name"], s["dept"], s["email"], ";".join(s["events"])])


def write_events(folder, by_event):
    Path(folder).mkdir(parents=True, exist_ok=True)
    for event, members in sorted(by_event.items()):
        with open(Path(folder) / event_file_name(event), "w", encoding="utf-8") as f:
            f.write(f"{event} - {len(members)} participants\n")
            for reg, name in sorted(members):
                f.write(f"{reg}  {name}\n")


def write_errors(path, problems):
    with open(path, "w", encoding="utf-8") as f:
        for where, kind, text in problems:
            f.write(f"{kind:<10}{where}: {text}\n")


def write_summary(path, stats, by_event, problems):
    kinds = {}
    for _, kind, _ in problems:
        kinds[kind] = kinds.get(kind, 0) + 1
    with open(path, "w", encoding="utf-8") as f:
        f.write("FEST REGISTRATION SUMMARY\n")
        f.write(f"Department files expected : {stats['expected']}\n")
        f.write(f"Department files missing  : {stats['missing']}\n")
        f.write(f"Lines read                : {stats['lines']}\n")
        f.write(f"Accepted students         : {stats['accepted']}\n")
        for kind in ("DUPLICATE", "EMAIL", "DAMAGED", "MISSING"):
            f.write(f"Rejected - {kind:<10}     : {kinds.get(kind, 0)}\n")
        f.write("Students per department:\n")
        for dept, n in sorted(stats["per_dept"].items()):
            f.write(f"  {dept:<6}{n}\n")
        f.write("Participants per event:\n")
        for event, members in sorted(by_event.items()):
            f.write(f"  {event:<14}{len(members)}\n")
```

```python
"""The whole flow: read -> validate -> remove duplicates -> write."""
from pathlib import Path

from . import reader, validator, writer
from .errors import InvalidRegistration

DEPARTMENTS = ["CSE", "ECE", "EEE", "MECH", "CIVIL"]


def run(input_dir, output_dir, departments=DEPARTMENTS):
    """Process the five department files. Never raises because of a bad line or a missing file."""
    out = Path(output_dir)
    out.mkdir(parents=True, exist_ok=True)
    seen = set()                        # register numbers already accepted: O(1) lookup
    students, problems, by_event = [], [], {}
    stats = {"expected": len(departments), "missing": 0, "lines": 0, "accepted": 0, "per_dept": {}}
    for dept, lines, problem in reader.read_all(input_dir, departments):
        if problem:
            stats["missing"] += 1
            problems.append((f"{dept}.txt", "MISSING", problem))
            continue
        for number, text in lines:
            if text.strip() == "":
                continue
            stats["lines"] += 1
            where = f"{dept}.txt line {number}"
            try:
                s = validator.parse_registration(text)
            except InvalidRegistration as err:
                problems.append((where, err.kind, f"{err.reason} | {text.strip()}"))
                continue
            if s["reg"] in seen:
                problems.append((where, "DUPLICATE", f"{s['reg']} already registered | {text.strip()}"))
                continue
            seen.add(s["reg"])
            students.append(s)
            stats["accepted"] += 1
            stats["per_dept"][s["dept"]] = stats["per_dept"].get(s["dept"], 0) + 1
            for e in s["events"]:
                by_event.setdefault(e, []).append((s["reg"], s["name"]))
    writer.write_master(out / "master_list.csv", students)
    writer.write_events(out / "events", by_event)
    writer.write_errors(out / "errors.log", problems)
    writer.write_summary(out / "summary_report.txt", stats, by_event, problems)
    return stats, problems
```

```python
"""Run the fest registration desk:  python3 run_fest.py <input folder> <output folder>"""
import sys
from festdesk import run

if __name__ == "__main__":
    src = sys.argv[1] if len(sys.argv) > 1 else "fest_input_clean"
    dst = sys.argv[2] if len(sys.argv) > 2 else "output_clean"
    try:
        stats, problems = run(src, dst)
    except OSError as err:          # only when the OUTPUT folder cannot be written: stop with a clear message
        print(f"Cannot write the output files to {dst!r}: {err}")
        sys.exit(1)
    print(f"{src} -> {dst}: accepted {stats['accepted']}, problems {len(problems)}")
```

Demonstration: demo_cases.py runs the package on all six input folders and prints, for each, the counts, the error log and the event files made. The six cases are: clean files, a duplicate registration, invalid e-mails, a damaged line (and blank line), a missing department file, and everything at once.

```
Terminal - Python 3.10.12

$ cd Task06
$ python3 make_fest_samples.py && python3 demo_cases.py
sample folders created
================================================================================
CASE clean: five clean files
  files expected 5, missing 0, lines read 15, accepted 15
  errors.log:  (empty)
  event files: dance.txt, drama.txt, photography.txt, quiz.txt, singing.txt
================================================================================
CASE duplicate: duplicate registrations
  files expected 5, missing 0, lines read 17, accepted 15
  errors.log:
    DUPLICATE CSE.txt line 4: 192311001 already registered | 192311001,Arun
    Kumar,CSE,arun.k@college.edu,Singing
    DUPLICATE ECE.txt line 4: 192311002 already registered | 192311002,bhavani
    s,ECE,bhavani@college.edu,Dance
  event files: dance.txt, drama.txt, photography.txt, quiz.txt, singing.txt
================================================================================
CASE bad_email: invalid e-mail addresses
  files expected 5, missing 0, lines read 15, accepted 12
  errors.log:
    EMAIL     CSE.txt line 2: e-mail must contain exactly one @ | 192311002,bhavani
    s,CSE,bhavani.college.edu,Singing
    EMAIL     EEE.txt line 1: domain has no dot | 192331001,gokul n,EEE,gokul@college,Photography
    EMAIL     MECH.txt line 3: e-mail must contain exactly one @ | 192341003,latha
    k,MECH,latha@@college.edu,Photography;Singing
  event files: dance.txt, drama.txt, photography.txt, quiz.txt, singing.txt
================================================================================
CASE damaged: damaged and blank lines
  files expected 5, missing 0, lines read 17, accepted 15
  errors.log:
    DAMAGED   ECE.txt line 2: expected 5 fields, found 3 | 192321009,only three fields,ECE
    DAMAGED   CIVIL.txt line 5: no event chosen | 192351004,pooja h,CIVIL,pooja@college.edu,
  event files: dance.txt, drama.txt, photography.txt, quiz.txt, singing.txt
================================================================================
CASE missing: EEE.txt is missing
```

```
Terminal - Python 3.10.12

  files expected 5, missing 1, lines read 12, accepted 12
  errors.log:
    MISSING   EEE.txt: department file is missing
  event files: dance.txt, drama.txt, photography.txt, quiz.txt, singing.txt
================================================================================
CASE all_problems: everything wrong at once (and CIVIL.txt missing)
  files expected 5, missing 1, lines read 14, accepted 11
  errors.log:
    EMAIL     CSE.txt line 3: domain has no dot | 192311003,charan p,CSE,charan.p@college,Quiz
    DUPLICATE ECE.txt line 4: 192311002 already registered | 192311002,bhavani
    s,ECE,bhavani@college.edu,Dance
    DAMAGED   MECH.txt line 2: expected 5 fields, found 1 | garbage line
    MISSING   CIVIL.txt: department file is missing
  event files: dance.txt, drama.txt, photography.txt, quiz.txt, singing.txt
```

Contents of some real output files: the first lines of master_list.csv (clean case), the quiz participant file, and the summary report of the 'everything at once' case:

```
Terminal - Python 3.10.12
$ cd Task06
$ head -n 4 output_clean/master_list.csv; echo; cat output_clean/events/quiz.txt; echo; cat
    output_all_problems/summary_report.txt
register,name,department,email,events
192311001,Arun Kumar,CSE,arun.k@college.edu,Dance;Quiz
192311002,Bhavani S,CSE,bhavani@college.edu,Singing
192311003,Charan P,CSE,charan.p@college.edu,Quiz;Photography

Quiz - 6 participants
192311001  Arun Kumar
192311003  Charan P
192321002  Elango M
192331003  Imran A
192341002  Karthik L
192351002  Nisha G

FEST REGISTRATION SUMMARY
Department files expected : 5
Department files missing  : 1
Lines read                : 14
Accepted students         : 11
Rejected - DUPLICATE      : 1
Rejected - EMAIL          : 1
Rejected - DAMAGED        : 1
Rejected - MISSING        : 1
Students per department:
  CSE   2
  ECE   3
  EEE   3
  MECH  3
Participants per event:
  Dance         4
  Drama         3
  Photography   2
  Quiz          4
  Singing       4
```

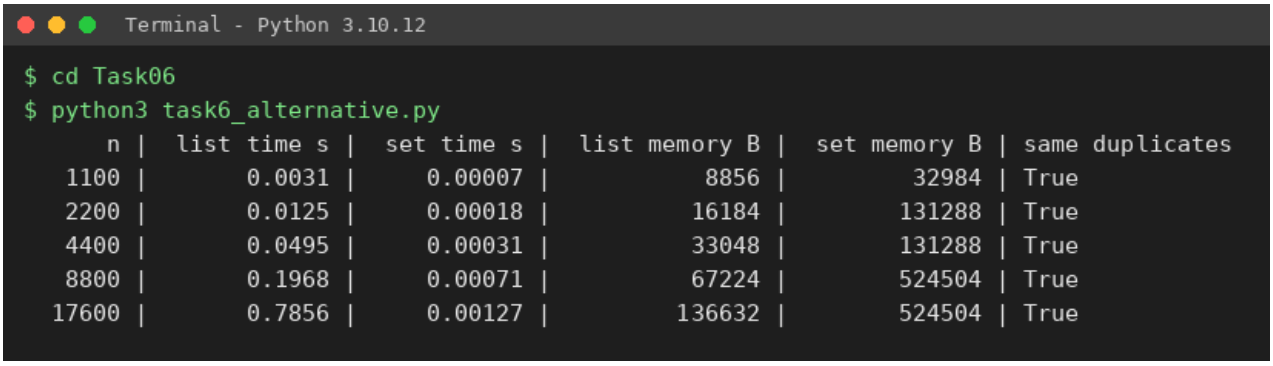
7. Analysis

- Clean files (R1, R7, R8): 15 lines read, 15 accepted, nothing in the error log. quiz.txt says '6 participants' and lists 6 students, one from each of the CSE (2), ECE, EEE, MECH and CIVIL files, which matches the 6 lines under it.
- Duplicate (R4): 17 lines read, 15 accepted, 2 DUPLICATE lines in the log: one repeated inside CSE.txt (line 4) and one student who registered again in ECE.txt (line 4) after already being in CSE.txt. The first registrations stayed in the master list.

- Invalid e-mail (R3): three different mistakes were caught with three different reasons: no @ (bhavani.college.edu), no dot in the domain (gokul@college) and two @ (latha@@college.edu). 12 accepted.
- Damaged line (R2, R11): a line with only 3 fields and a line with no event were logged as DAMAGED with line numbers (ECE line 2, CIVIL line 5). The blank line at the top of CIVIL.txt was skipped without an error, and it also moved the damaged line down to line 5, which is the number the log gives.
- Missing department file (R5, R6): EEE.txt was not there; the log has one MISSING line, the summary says 1 of 5 files missing, and the other four departments were fully processed (12 accepted).
- Everything at once: CIVIL.txt missing, one duplicate, one bad e-mail and one garbage line were all logged with the right kind (4 problems) and 11 students were accepted. The program finished normally in all six runs (R6).

8. Alternative Concept and Complexity Analysis

Alternative: detect duplicates by searching a list of all earlier registrations (if reg in earlier_list) instead of keeping a set of register numbers already seen. I wrote both (task6_alternative.py) and timed them for n registrations, all with different register numbers (the worst case for the list, where every search goes to the end) plus a few repeats. Both found the same number of duplicates every time.



Version	Time for n registrations	Extra space	Measured: time when n doubles
list of earlier registrations	O(n^2): each of the n lookups scans up to n items	O(n) for the list	about 4 times longer (0.0495 s to 0.1968 s to 0.7856 s)
set of register numbers (my design)	O(n) on average: each lookup is O(1) on average	O(n) for the hash table (larger than the list's pointer array)	about 2 times longer (0.00031 s to 0.00071 s to 0.00127 s)

Table T6-4: complexity. The sizes shown are for n = 4,400, 8,800 and 17,600 registrations.

The list version took 0.0031 s for 1100 registrations and 0.7856 s for 17600: the number of registrations went up 16 times and the time went up about 253 times, which is what O(n^2) predicts (16 x 16 = 256). The set version went from 0.00007 s to 0.00127 s, about 18 times, close to the 16 times that O(n) predicts, so for 17,600 registrations it is about 619 times faster. The price is space: both are O(n), but the set (a hash table) stores more per item; the measured size of the set object was 524,504 bytes against 136,632 bytes for the list at the largest size (these sizes do not include the strings themselves, which both versions hold). For a college fest with a few thousand registrations the list would still be fast enough, but the set is as simple to write and does not get slower per registration as the event grows.

9. Conclusion

I turned the problem statement into 11 testable requirements, drew the flow, decided where each exception is handled, and built a package with one module for each job. The six demonstrations showed every required case (clean, duplicate, invalid e-mail, damaged line, missing file, all together) with the program finishing normally each time and every rejected line in errors.log with its file name and line number. Using a set for duplicate detection makes that step O(n) instead of O(n^2).